Satchel

User Handbook

Verified against commit ffe1b0a · July 2026

Contents

Copyright and disclaimer

Satchel — User Handbook Verified against commit `ffe1b0a` · July 2026

This handbook tracks the code by commit hash rather than a release version: the hash above is the `satchel` source revision its contents were verified against. When the code moves, the hash is bumped and the affected pages updated.

> **Disclaimer** — Satchel is software for self-custody trading of digital assets by means of atomic swaps. The project does not operate an exchange, does not take custody of funds, does not match or execute orders on your behalf, and provides no investment advice or guarantees of any kind. You alone hold your keys and your recovery phrase, and you alone are responsible for the security of your funds and the machine on which the software runs. The software is provided "as is", without warranty of any kind; the information in this handbook is provided in good faith and may contain errors or omissions. Consult the official project repository for the most up-to-date information.

Part I

Getting Started

Chapter 1

About this Handbook

Welcome. This is the **Satchel User Handbook** — a friendly, step-by-step guide to trading cryptocurrency directly with another person, with no exchange in the middle and no one ever holding your coins but you.

If that sounds technical, don't worry. This handbook assumes **no background in cryptocurrency**. Wherever a new idea comes up — a *swap*, a *seed*, a *recovery phrase* — we explain it the first time you meet it, in plain language, before moving on. You will need to be comfortable installing a desktop application and following on-screen steps; everything beyond that, we walk you through.

1.1 Who this is for

This handbook is for **anyone who wants to trade one cryptocurrency for another without trusting a middleman**. You might be:

- Someone curious about *trustless trading* — swapping coins peer-to-peer in a way where the blockchain itself enforces the deal, so neither side can run off with the other's money.
- Someone who already holds Bitcoin or Bitcoin-PoCX and wants to trade between them safely, on your own machine, on your own terms.
- A member of a community that runs its own trading noticeboard and wants to use it.

You do **not** need to know how Bitcoin works under the hood, what a "transaction" is, or any programming. If you ever do want the deep technical detail — how the swap transactions are built, the exact protocol — that lives in a separate companion book, the *Pact Developer & Integrator Handbook*. This handbook points you there only when it is genuinely useful.

1.2 How this handbook is organised

The handbook is in four parts, moving from "getting set up" to "trading" to "the finer points":

1. **Getting Started** — what Satchel is, installing it, creating your trading identity, and connecting your coins. Start here. *(You are reading this part.)*

2. **Trading** — a tour of the app, browsing the **Corkboard**, posting an offer, taking someone else's, watching a swap complete, your wallet view, and private one-to-one offers.
3. **Transports & Network** — how your offers reach other people (over **Nostr** or a **Corkboard**), and the settings that control it.
4. **Safety & Reference** — keeping your funds safe, what really happens during a swap, troubleshooting, a glossary of terms, and where to get help.

You do not have to read straight through. If you just want to get trading, work through Part 1 once, then dip into Part 2 as you go.

1.3 Conventions used in this book

We use a few simple conventions throughout.

- **Callouts** flag things worth pausing on:

  > **Tip** — a shortcut or a friendlier way to do something.
  >
  > **Note** — background, a clarification, or a pointer to another chapter.
  >
  > **Warning** — something that could cost you funds if you get it wrong. Please read these carefully.

- **Text styles** carry meaning. **Bold** marks something you see on screen — a button, a field, a menu, or the name of a screen. `Monospace` marks anything you type exactly, a file path, or a web address. *Italics* introduce a new term the first time it appears.
- **Numbered steps** are procedures to follow in order. Bulleted lists are just lists.
- **Screenshots** show you what a screen looks like. Satchel is young and changing quickly, so a screenshot may differ slightly from what you see — the labels and the order of steps are what matter, and we keep those accurate.
- **Cross-references** name another chapter by its title — for example, *see the chapter "Setting Up Your Coins"* — rather than by number, because numbers shift as the book grows.

1.4 What this edition tracks

Rather than a release-version number, this handbook tracks the **source revision** it was checked against: it was verified against commit `ffe1b0a` (July 2026). The commit hash on the copyright page is the single status marker — when the code moves, the hash is bumped and the affected pages are updated.

> **Note** — The first trading pair is **BTCX** ↔ **BTC** (BTCX is Bitcoin-PoCX). Screens, labels, and details evolve with the code; where an exact detail matters, the app itself is the final word.

1.5 A word on safety

Trustless trading puts you in full control — which also means **you alone are responsible for your funds**. This is different from an exchange, where a company holds your coins (and can lose or freeze them). With Satchel, no one can freeze or seize your funds. The flip side is that no one can recover them for you either. Two rules carry almost all the safety you need:

> **Warning** — When you create your trading identity, Satchel shows you a *recovery phrase* — a list of 12 or 24 words. **Write it down on paper and keep it somewhere safe.** It is the only way to recover your funds if your computer is lost or breaks.
>
> **Warning** — **Never share your recovery phrase with anyone, and never type it into a website.** Anyone who has those words can take your funds. No genuine support person will ever ask for it. Satchel will never ask you to enter it on a web page.

We come back to safety in detail in the chapter *"Staying Safe"*, but those two rules are worth carrying with you from page one.

1.6 Where to get help

- The project's repository and website carry release notes, design notes, and the latest news.
- The chapter *"Troubleshooting"* covers the most common snags — a node that won't connect, an offer that won't post — with fixes.
- The chapter *"Getting Help"* lists the community channels where you can ask questions.
- If you want the technical "how it works" underneath Satchel, the companion *Pact Developer & Integrator Handbook* is the place to look.

Chapter 2

What Satchel Is

Satchel is a desktop app for **trading one cryptocurrency for another, directly with another person**. There is no company in the middle. No one takes custody of your coins, no one matches your trades, and there are no trading fees. You and your counterparty swap directly, and the blockchain itself makes sure neither of you can cheat.

This chapter explains, in plain language, what that means and how the pieces fit together. You don't need any of it to start trading — but it helps to know what is actually happening to your money.

2.1 Trading without a middleman

On a normal exchange, you hand your coins to the exchange, it holds them, and it arranges trades on your behalf. You are trusting that company not to lose your funds, not to freeze your account, and not to disappear. Plenty have.

Satchel works differently. It uses an *atomic swap* — a way of trading two coins so that the exchange happens **all at once or not at all**. Either both sides get what they agreed to, or both sides keep what they started with. There is no in-between where one person has paid and the other has run off. The word *atomic* just means "indivisible": the trade cannot be left half-finished.

Because the blockchain enforces this, you never have to trust the other person — and you never have to trust Satchel with your coins either. This is what we mean by *trustless*: not "no one is trustworthy", but "you don't need to trust anyone for your money to be safe".

What this gives you:

- **No custody.** Your coins stay under your control the entire time. Satchel never holds them.
- **No exchange, no account.** There is nothing to sign up for and no company that can freeze you out.
- **No fees.** Satchel takes nothing. You pay only the ordinary network fee that every blockchain charges to process a transaction — the same fee you'd pay sending coins to a friend.

- **No matching engine.** Nobody decides who trades with whom. You browse offers and pick the one you like.

2.2 The village market

Satchel borrows its names from an old-fashioned village market square, because the picture is a good one.

- A *pact* is the **deal** — the agreement to swap so much of one coin for so much of another. (Behind the scenes, "Pact" is also the name of the engine that carries out the deal; more on that below.)
- The *corkboard* is the **noticeboard** in the square where people pin up their offers for everyone to see. You browse it, and when you see a deal you like, you take it.
- Your *satchel* is your **bag** — where your trades settle and where you keep track of what you've done.

So in one sentence: a *pact* is posted on the *corkboard*, and settled into your *satchel*. You will see all three words in the app, and now you know what each one means.

2.3 The three moving parts

Under the hood, Satchel has three pieces. The important thing to understand is a single hard rule: **your coins and your keys never leave your own machine.**

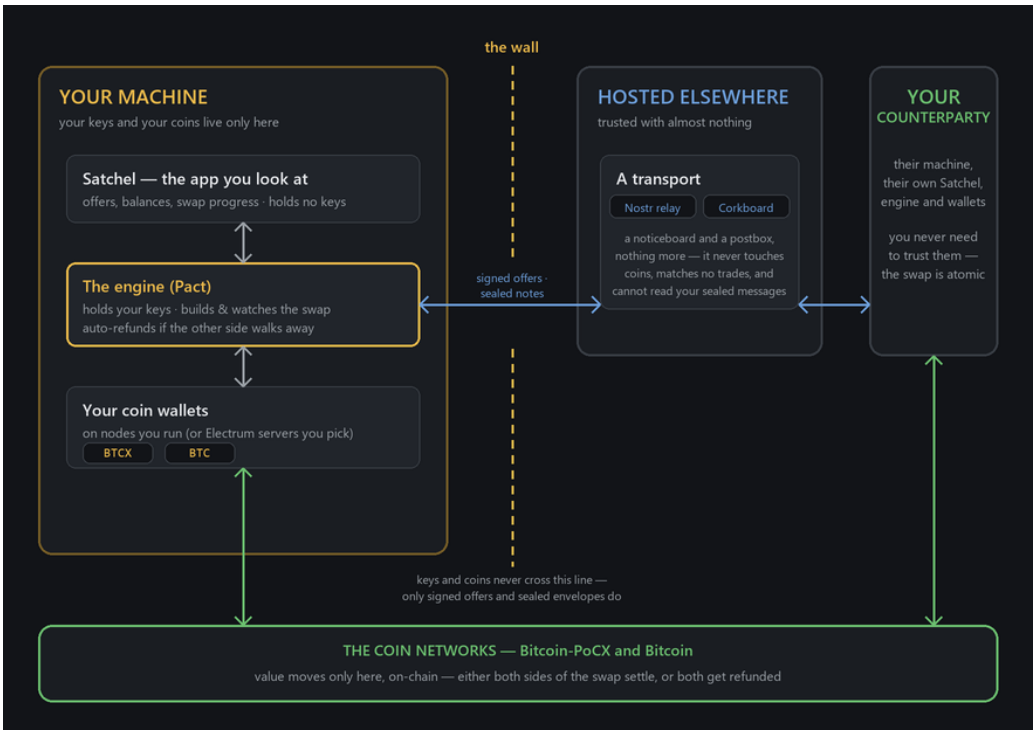

Figure 2.1: How the parts fit together: everything that touches your coins runs on your machine; the hosted side only carries signed notes.

On your machine (and only here):

1. **Satchel** — the app you look at. It shows you offers, balances, and the progress of your swaps. It is the friendly face; it does not hold your keys.
2. **The engine** — the part that does the real work of a swap. It holds your keys, builds and watches the swap transactions, and — importantly — automatically gives you your money back if the other side walks away. (Its proper name is *Pact*; the app simply calls it "the engine".)
3. **Your coin nodes** — your own copies of the Bitcoin-PoCX and Bitcoin networks. Satchel is a *node-backed* app, which means it trades using wallets that live on nodes you run. We cover setting these up in the chapter *"Installing Satchel"* and connecting them in *"Setting Up Your Coins"*.

Hosted elsewhere (and trusted with almost nothing):

4. A **transport** — either a **Nostr** relay or a **Corkboard** — that simply carries your offers and a few sealed coordination messages between you and your counterparty. That's all it does. It never touches your coins, never matches trades, never charges anything, and cannot read your private messages. It is a noticeboard and a postbox, nothing more.

   Note — This wall is the whole point. Everything that could touch your money runs on your computer. The hosted side sees only signed offers and sealed envelopes — never your coins, never your keys. We come back to it in the chapter *"Transports"*.

2.4 What you can trade today

The first supported pair is **BTCX** ↔ **BTC** — that is, Bitcoin-PoCX traded against Bitcoin. *BTCX* is the ticker for Bitcoin-PoCX. More coins will follow, and the app is built so new coins can be added without a new release.

2.5 Setting expectations

Satchel is live and released. The underlying swap protocols (both the original style and the newer, more private Taproot style) have been reviewed. The design is careful and the safety guarantees are real — but it is software handling real value, and you alone hold your keys.

> **Warning** — Treat your funds with care: above all, follow the safety rules in the chapter *"Staying Safe"* — write down your recovery phrase, and never share it. Your keys and recovery phrase are yours alone to protect.

Chapter 3

Installing Satchel

Installing Satchel is a two-part job. The app itself is a single download. But because Satchel trades using **your own coin nodes**, you also need those nodes running before you can trade. This chapter walks through both, gently.

3.1 What you need first: your own coin nodes

Satchel is a *node-backed* app. A *node* is your own personal copy of a cryptocurrency's network — the software that holds a wallet, knows the current state of the blockchain, and can send and receive coins. When you swap, Satchel uses the wallets on your nodes to fund your side of the trade and to receive the proceeds.

To trade the first pair, you need **two** nodes running on your machine (or on a machine Satchel can reach):

- **Bitcoin-PoCX** (BTCX) — a `bitcoin-pocx` node.
- **Bitcoin** (BTC) — a standard Bitcoin Core node, or anything compatible with it.

Each node must have its **RPC interface reachable**. *RPC* (Remote Procedure Call) is simply the doorway through which other programs — like Satchel's engine — talk to your node. In practice this means the node is running and configured to accept local connections. You will point Satchel at each node's address and port later, in the chapter *"Setting Up Your Coins"*; for now, the goal is just to have both nodes installed and running.

> **Note** — Don't worry about the exact RPC settings yet. The chapter *"Setting Up Your Coins"* covers the connection form field by field, including how Satchel reads your node's login automatically from its *cookie file* so you usually don't have to type a password.
>
> **Tip** — You need at least **two** live coins before Satchel will let you trade, because a swap involves two chains. If you only have one node up, that's fine for installing — you'll just finish connecting the second before your first trade.

3.2 Getting the app

Satchel is distributed as a ready-to-run *bundle* — a single download containing everything the app needs.

1. Go to the project's **releases** page.
2. Download the bundle for your operating system:
   - **Windows** — the Windows installer, `Satchel-<version>-Windows-x64-Setup.exe`. Run it, and it installs Satchel for you.
   - **Linux** — pick the format your distribution likes: the AppImage (`Satchel-<version>-Linux-x64.AppImage`, a single self-contained file you can run directly), or the `.deb` / `.rpm` package (`Satchel-<version>-Linux-x64.deb` / `.rpm`).
3. Run the installer (or the AppImage), then launch the **Satchel** application.

That's the whole installation. There is nothing to register and no account to create.

On **Windows**, the installer also tucks the engine's command-line tools — `pact-cli` and `pactd` — onto your **user PATH**, so you can run them from any terminal. You won't need them for ordinary trading, but if you do want them, **open a new terminal** afterwards: an already-open window won't see the change until it's reopened.

> **Note** — **macOS is not supported yet.** A macOS build is planned. For now, use Windows or Linux.
>
> **Warning** — If you installed an **earlier release-candidate build** of Satchel, a now-fixed installer bug could have truncated your Windows user PATH, dropping unrelated entries past a certain length (so a tool like `cargo` might suddenly seem to have vanished from the command line). Current builds fix this. If something went missing, just **re-add it once** — for example, add `%USERPROFILE%\.cargo\bin` back to your PATH — and it'll stick.
>
> **Note** — On **Windows**, installing or uninstalling a new version now stops any `pactd`/`pact-cli` still running **from this install's own folder** first. This matters if you chose **Keep running** on a previous quit (see the chapter "Tracking Your Swaps") and the engine was still alive in the background when you upgraded: without this, the running engine would hold a file lock through the upgrade, or the new Satchel could end up re-adopting the *old* engine binary instead of the freshly installed one. If a stop like this ever interrupts a live swap mid-step, that's safe — the engine persists its state around every broadcast, and the freshly installed `pactd` picks the swap back up via chain-watching as soon as it starts. Only daemons running from *this* install's folder are ever touched; a developer build or a playground instance running elsewhere on the same machine is untouched.
>
> **Warning** — **Upgrading from rc9 to rc10? Settle or abort any live Private (Taproot) swap first.** rc10 changes how Private swaps are co-signed, at the protocol level, so an rc9 peer and an rc10 peer **cannot open a Private swap with each other** — the handshake simply fails cleanly, before any funds move, with nothing at risk.

Standard (HTLC) swaps are unaffected. Once you and your counterparties are on rc10, Private swaps work as before.

3.2.1 What's inside the bundle

The bundle contains two things working together:

- **Satchel** — the app you interact with.
- **The bundled engine** — Satchel ships *Pact*, the swap engine, alongside itself and starts it for you automatically. You don't install or run it separately; Satchel launches it, supervises it, and shuts it down when you're done.

So a single download gives you both the face and the engine. The only thing you provide is your coin nodes.

> **Note** — On **Windows**, Satchel keeps its merchants, your recovery seed, and your settings under `%LOCALAPPDATA%` (your machine-local app-data folder), not the roaming `%APPDATA%`. This is deliberate: a spending seed should stay tied to this one machine and never roam to others. You don't have to do anything with this — it's just where Satchel stores things.

3.3 Windows: the SmartScreen prompt

Satchel's builds are **not yet code-signed** — signing is a paperwork step the project is still completing. Because of this, Windows **SmartScreen** may show a blue warning the first time you run the app, saying something like *"Windows protected your PC"*.

This is expected for unsigned software and does not mean anything is wrong. To run it:

1. Click **More info** in the SmartScreen dialog.
2. Click **Run anyway**.

Satchel will then start normally.

> **Warning** — Only do this for a bundle you downloaded from the **official** releases page. Treat any copy of "Satchel" from somewhere else with suspicion — a tampered build could put your funds at risk. When in doubt, download fresh from the official source.

3.4 First-time prerequisites

On a current, up-to-date copy of **Windows**, you generally need nothing extra: Satchel uses a component called **WebView2** to draw its window, and WebView2 already ships with modern Windows. If for some reason it is missing, Windows will offer to install it.

On **Linux**, the bundle is self-contained; follow any notes on the releases page for your distribution.

Beyond that, the only real prerequisite is the one from the top of this chapter: **your two coin nodes, up and reachable.**

3.5 For developers: building from source

If you'd rather build Satchel yourself from the source code, that path is for developers and is documented elsewhere:

- The project **`README`** covers the prerequisites (Rust, Node, the Tauri CLI) and the build commands.
- The companion *Pact Developer & Integrator Handbook* covers the engine in depth.

  Note — One quirk of running a development (unpackaged) build: **Windows often suppresses its desktop notifications**, because it only reliably shows toasts for installed apps. The installed Satchel notifies normally; the test button under **Settings → Notifications** says as much if a test toast doesn't appear.

Most people don't need this — the prebuilt bundle above is all it takes to start trading. With Satchel installed, the next chapter walks you through your very first launch.

Chapter 4

First Launch & Setup

The first time you open Satchel, it walks you through a short setup, one screen at a time, like a friendly wizard. You can't skip ahead and you can't get it wrong by accident — each step unlocks the next. This chapter follows that sequence in order:

1. Create (or import) a **merchant** — your trading identity.
2. Back up your **recovery phrase**, then choose whether to protect it with a passphrase.
3. (On later launches only) **unlock** if you chose a passphrase.

After that, Satchel checks your coins are connected and hands off to the next chapter.

> **Tip** — Prefer to do this in another language? Look for the **globe** menu in the **top-right corner of the welcome dialog**. Click it and pick from the list — Satchel ships in **26 languages**, and the choice applies right away, so you can switch before or even partway through setup. The same picker is always available later from the header.

4.1 Step 1 — Create your merchant

The first screen welcomes you and explains that Satchel trades under a *merchant*. A merchant is simply **your trading identity**. Behind that one word sits a tidy rule:

> **Note** — **One merchant = one seed = one data folder.** A *seed* is the master secret that all your keys come from (more on it in the next step). Each merchant has its own seed, its own swap history, and its own folder on disk — much like separate wallets. Keeping trades under different merchants keeps them from being linked together, which is good for privacy.

You have two choices on this screen:

- **Create new** — make a brand-new identity with a fresh seed. Choose this if you're starting out.
- **Import** — restore an identity you already have, using its recovery phrase. Choose this if you've used Satchel before and want to bring an existing merchant onto this machine.

Let's create a new one.

1. Click **Create new**.
2. Satchel asks for a **Merchant name** — just a friendly label so you can tell your merchants apart (for example, `Main`). It's only for your own eyes, and you can leave it blank — Satchel falls back to a default label.
3. Click to continue. Satchel moves you straight to the seed step. Nothing has been created yet — the merchant only comes into being at the wizard's final step, so backing out anywhere before that leaves no trace.

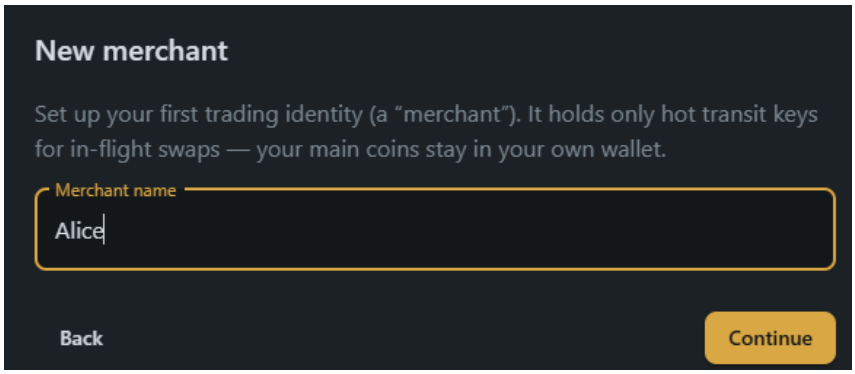

Figure 4.1: Naming your first merchant.

4.2 Step 2 — Your recovery phrase

This is the single most important step in the whole book, so we'll take it slowly.

4.2.1 What a recovery phrase is

Satchel now generates a *recovery phrase* (also called a *seed phrase* or *mnemonic*) — a list of **12 or 24 ordinary words**, like `ripple cabin lunar quote` …. Those words are a human-readable backup of your seed, the master secret behind every key your merchant uses.

> **Warning** — Anyone who has these words **controls this merchant's funds.** Satchel keeps no copy you can recover from — if you lose the words and lose your computer, the funds are gone for good. This is the trade-off for being your own bank.

4.2.2 Write it down

Satchel shows you the words on a **"Write down your recovery phrase"** screen, numbered in order. A toggle above the words lets you choose a **12- or 24-word** phrase — the default is **12**, and 12 is plenty: this is a hot transit wallet, not cold storage. Pick 24 if you prefer the longer phrase. Switching the toggle generates a fresh phrase, so settle on a length *before* you start writing.

1. Get a pen and paper. (Paper, not a photo and not a text file — a backup that lives only on a device can be lost or stolen along with it.)
2. Write the words down **in order**, with their numbers.
3. Double-check what you wrote against the screen.

4. Tick the box **“I have written down my recovery phrase.”**
5. Continue.

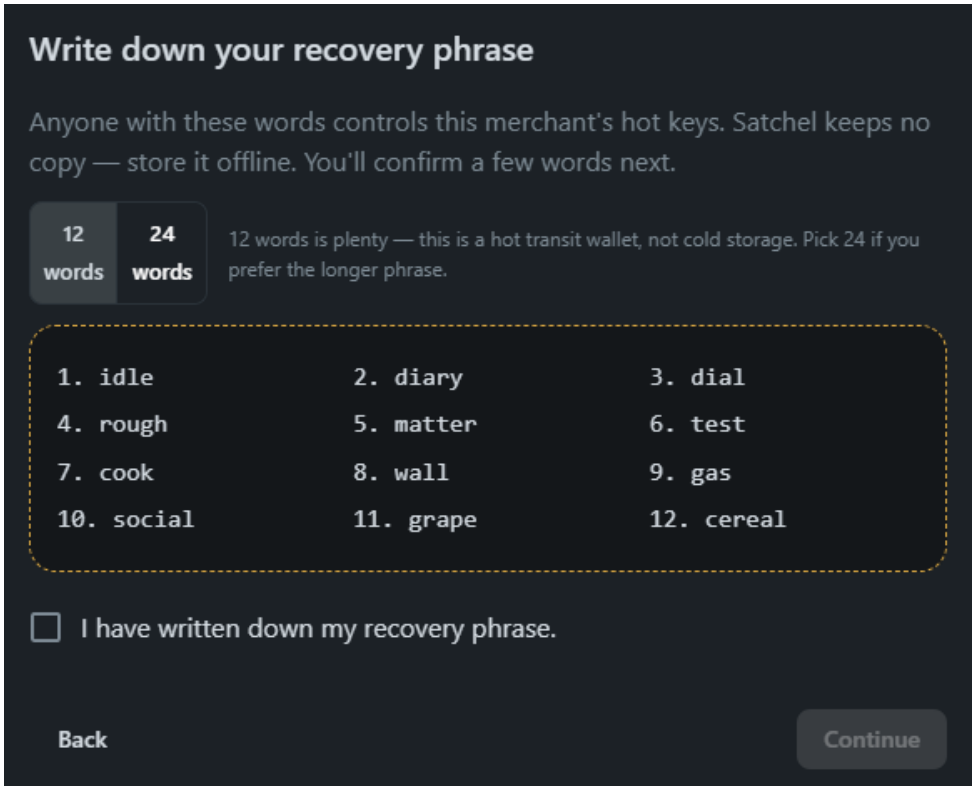

Figure 4.2: The recovery phrase, shown for you to copy down.

> **Warning** — **Never share these words with anyone, and never type them into a website.** No genuine support person will ever ask for them. Satchel itself will never ask you to re-enter your phrase on a web page.

4.2.3 Confirm you wrote it down

To make sure your backup is good, Satchel next asks you to **confirm a few words** — it names a couple of positions (for example, “Word #4”) and you type the matching words back. As you type, it offers the valid words so a typo can’t slip through.

If a word doesn’t match, Satchel tells you so you can check your written copy before going on. This little check has saved many people from a backup with a wrong word in it.

> **Note** — On a practice (regtest) network, this confirmation step is skipped, because there’s no real money at stake. On the real network you’ll always be asked to confirm.

4.2.4 Importing instead

If you chose **Import** back in Step 1, this step looks a little different: rather than revealing words, Satchel gives you a **numbered grid of word cells** — arranged in three columns, one cell per word — to **enter your existing 12- or 24-word phrase**. A toggle at the top lets you say whether your phrase is **12 or 24 words**, so the grid has exactly the right number of slots.

Type into the cells one word at a time, and Satchel **suggests the matching word** as you go (the words come from a fixed list, so it can autocomplete each one). Any cell holding a word that isn't on that list is **highlighted** so you can spot a typo at a glance. In a hurry? You can **paste your whole phrase** into the first cell and Satchel spreads the words across the grid for you.

As you fill it in, a **status line** at the bottom keeps you posted:

- "*Enter all 12 words.*" (or 24) while the grid isn't full yet.
- "*Some words aren't in the BIP39 wordlist — check the highlighted ones.*" if a cell holds a word it doesn't recognise.
- "*Checksum doesn't match — re-check your words and their order.*" if every word is valid but the phrase as a whole doesn't add up — usually a word in the wrong place.
- "*Recovery phrase looks valid.*" once everything checks out.

Only when you see **"Recovery phrase looks valid."** can you continue — Satchel won't let you import a phrase it can't verify, which protects you from a small slip costing you the import. Then you carry on to the passphrase choice, the same as everyone else.

4.3 Step 3 — Passphrase: protect the seed, or not

Last, Satchel asks how to store your seed on disk. Two cards:

- **No passphrase (recommended)** — you never have to type anything to start trading, and Satchel's automatic safety net (the part that gives you your money back if a swap stalls) keeps working even after a reboot, with nothing for you to do. The seed still isn't left as plain text on disk: on Windows and macOS Satchel locks it with a key kept in your operating system's secure store, so the file alone is useless if copied to another machine. That key lives on this computer, though, so it doesn't protect you from someone who is already signed in as you — for that, choose a passphrase below.
- **Encrypt with a passphrase** — the seed is locked with a passphrase you choose. It's safer if someone gets access to your computer's files, but there's a cost: you must re-enter the passphrase each time you start Satchel, and the automatic safety net stays paused until you do.

  Note — Satchel **never stores your passphrase.** If you encrypt and then forget the passphrase, no one can unlock the seed for you — your recovery phrase is the only way back in. Either way, your written recovery phrase remains your real backup.

  Tip — The seed in a merchant is a *hot* seed — it holds only the temporary keys needed to move coins through a swap, not a long-term vault. For most people, **No passphrase** is the right, convenient choice. Whichever you pick, sweep any sizeable proceeds out to your own main wallet rather than letting them pile up here.

Make your choice and finish. This is the moment your merchant is actually created — name, seed, and storage choice are committed together in one step. Everything before it was just collecting your answers, so cancelling at any earlier screen leaves nothing half-made behind

(and if you were adding a second merchant later, the one you were using stays active). Your merchant is ready.

4.4 Later launches: unlocking

If you chose **No passphrase**, Satchel goes straight to trading every time you open it.

> **Note** — One rare exception: if this computer can no longer unlock the seed stored on disk — typically because the data folder was moved to a new machine, or the system keychain was reset — Satchel opens a **guided re-import dialog** at startup explaining exactly that. Re-enter your recovery phrase and you're back; **your funds and swaps are not affected**. The *Backup, Seeds & Safety* chapter has the details.

If you chose **Encrypt with a passphrase**, the next time you start Satchel you'll see an **Unlock merchant** screen. Type your passphrase to unlock the seed for this session. Satchel holds it in memory only and forgets it when you close the app — it is never written down. From this screen you can also switch to a different merchant instead.

4.5 Managing more than one merchant

You're not limited to one identity. Later, from the merchant control at the top of the window, you can **create or import more merchants** and **switch** between them — each with its own seed, history, and folder. This is handy for keeping, say, a main identity and a separate one for private trades. We cover the **Merchant Manager** in Part 2.

> **Note** — Satchel won't let you switch away from a merchant that has a swap in progress — that's a safety measure, so an in-flight trade is never abandoned.

4.6 What's next

With your merchant created and your seed backed up, Satchel drops you straight into the app. On this first run it offers a **one-time, skippable coin-setup dialog** to help you connect two coins — but it never blocks: click **Later** to go straight to browsing with zero coins configured, or set them up now. It won't appear again once dismissed. Connecting coins is exactly what the next chapter, *"Setting Up Your Coins"*, covers.

Chapter 5

Setting Up Your Coins

A swap always involves **two** blockchains — you give one coin and receive another. So before Satchel will let you *trade*, it needs **at least two coins connected and live**. This chapter walks you through connecting them.

You don't have to set anything up just to look around, though. Satchel drops you straight into the app after you unlock, and you can **browse the Corkboard** with zero coins configured — the board shows every pair on offer. Coin setup is what turns browsing into trading. Reach it any time from **Settings** → **Coins**.

> **Note** — *Live* means a coin is connected **and** its node is up and answering. Trading is gated **per action**, not by an app-wide wall: with fewer than two live coins the **Post an offer** and **Create slip** screens show a soft "Set up two coins to trade" prompt, and a board offer's **Take** button stays disabled until both of its coins are live. That's the safety gate: no trading until you genuinely have two working chains — but you can look around freely until then.

5.1 The coins screen

Open **Settings** → **Coins** and you'll see a card for each coin Satchel knows about. The first pair is **BTCX** (Bitcoin-PoCX) and **BTC** (Bitcoin). Each card shows:

- The coin's **icon, name, and symbol**.
- A **status pill** — *Not set up*, *Connected* (with the node's current block height, called the *tip*), or *Connection error*.
- Once configured, a **connection chip** — **RPC (local)**, **RPC (remote)**, **Electrum (local)**, or **Electrum (remote)** — telling you at a glance how this coin connects and whether that connection stays on this machine (*local* means every host is loopback). Hover it for a plain-language explanation; an Electrum coin's chip is tinted green like its pact-seed wallet label, and its summary line shows the first server plus how many more back it up.
- **Capability chips** — small tags showing what the coin can do (more on these below).
- A **Set up** button (or **Edit connection** if it's already configured), and a way to remove it.

To connect a coin, click **Set up** on its card.

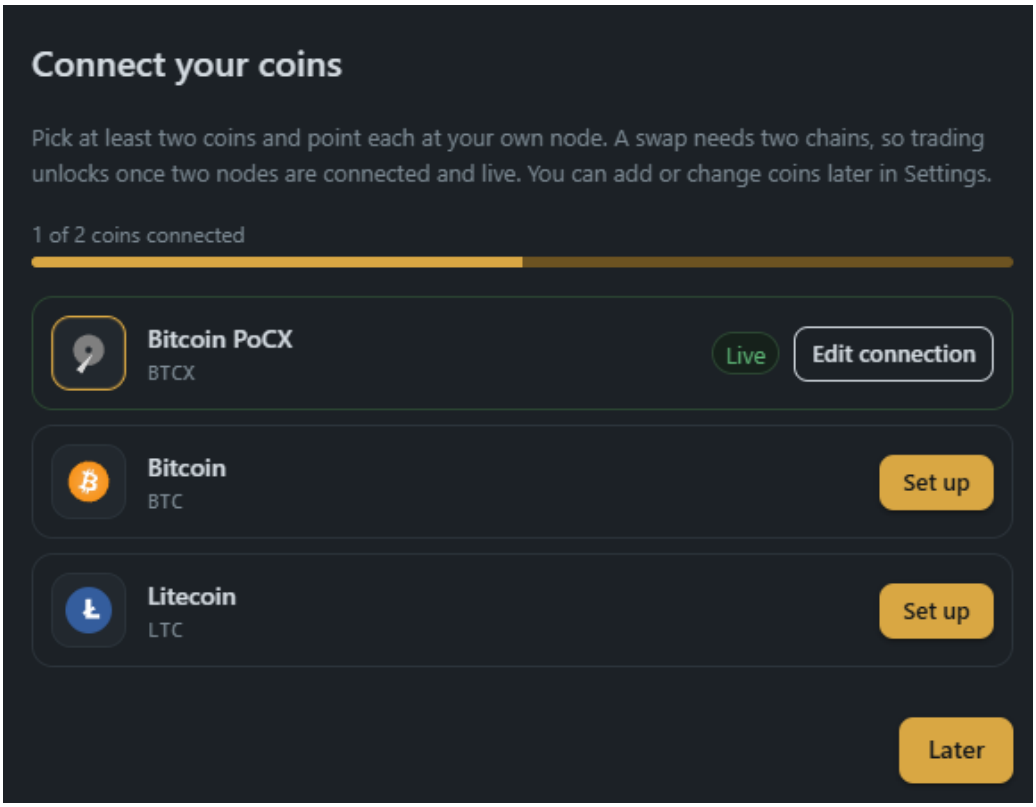

Figure 5.1: The Coins screen, with one coin connected and one still to set up.

5.2 Just want to look?

Maybe you're not ready to connect nodes yet — you'd just like to see what's trading before you commit. You don't have to do anything special: with no coins configured, Satchel still opens on the **Corkboard** and shows you the whole live board, every pair on offer. Browse all you like.

What you *can't* do until you connect coins is **post** a new offer, **take** someone else's, or **fund** a swap — anything that moves money needs **two live coins**. Those actions guide you back here rather than blocking the app: the **Post an offer** and **Create slip** screens show a soft "Set up two coins to trade" prompt, and a board offer's **Take** button stays disabled until both of its coins are live. When you're ready to trade for real, connect two coins as below.

5.3 The connection form

The first choice in the form is the **connection type** — where this coin's chain data and wallet should live. A **new** coin starts on **Electrum**, the quick no-node default; the two choices are:

- **Electrum** *(the default)* — no node at all. Chain data comes from Electrum servers and the wallet lives on **your Pact seed** (the servers never see your keys). The form becomes a single list of server URLs, one per line (`tcp://host:port` or `ssl://host:port`), pre-filled with the defaults shipped for the coin. On **mainnet at least two independent servers**

are required — they cross-check each other's view of the chain. *Validate servers* runs the same genesis check as for a node, plus a capability handshake (protocol version; pruned servers are refused). This is the quick path for a coin whose node you don't want to sync — say, Bitcoin, when you already run a BTCX node.

- **Your own node** — Core RPC. The node's wallet funds swaps; maximum sovereignty. The rest of this section describes its fields.

You can change a coin's connection type later — but one direction deserves a heads-up. Satchel keeps **one wallet per coin, never mixed**: an Electrum coin's funds live on your Pact seed, a node coin's funds live in the node's wallet. So if you switch an Electrum coin **back to node mode** while its pact-seed wallet still holds coins, Satchel warns you on **Save** — *"This hides your pact-seed wallet"* — and asks you to confirm with **Switch anyway**.

> **Warning** — The warning is about *visibility*, not loss: the coins **stay safe on your seed** and reappear the moment you switch back to Electrum. But until then they won't show up in Wallets or fund swaps. The tidy move is to **send them somewhere first** — your node wallet, say — and switch after. (If the Electrum servers are already unreachable — often the very reason you're switching — Satchel can't read the balance and lets the switch through without the prompt.)

Choosing **Your own node** means telling Satchel **where your node is** and **how to log in to it**. It's filled in with sensible defaults, so often you'll only change a field or two.

The fields, in order:

- **RPC host** — the address of the machine running the node. The default is `127.0.0.1`, which means "this same computer". If your node runs on another machine, put its address here.
- **RPC port** — the port number the node listens on for RPC. Each coin and network has its own usual port; the form is pre-filled with the expected one, but check it matches your node's configuration.
- **Authentication** — how Satchel proves to your node that it's allowed to connect. Pick one of two cards:
  - **Cookie file** *(the default)* — your node writes a small `.cookie` file containing a one-time login, and Satchel reads it automatically. Nothing to type, nothing to store. When you choose this, a **Node data directory** field appears — point it at your node's data folder, and Satchel finds the cookie inside (it shows you the exact path it will read). It comes **prefilled with the right default for your operating system** — `%LOCALAPPDATA%\<Node>` on Windows, `~/Library/Application Support/<Node>` on macOS, `~/.<node>` on Linux — so the cookie path is correct out of the box and Windows users no longer have to hand-fix it. Change it only if your node keeps its data somewhere custom.
  - **User / password** — if your node is set up with a fixed `rpcuser` and `rpcpassword` (common for a node on another machine), choose this and enter the **RPC username**

and **RPC password** from your node's config.

- **Wallet name** *(optional)* — if your node has more than one wallet loaded, name the one Satchel should use for this coin. Leave it blank to use the default.
- **Confirmations before final** *(optional)* — how many blocks deep a payment on this chain must be before Satchel treats it as settled. Leave it blank to use the recommended default for the coin. If you do set a number, the allowed range runs from **2** up to that recommended default — the default is also the maximum, and the form checks the range. Higher numbers within the range are a little safer against rare blockchain reshuffles, but make swaps slower. This setting is yours alone: your trading partner picks their own, and each side simply shows the other's choice while waiting.

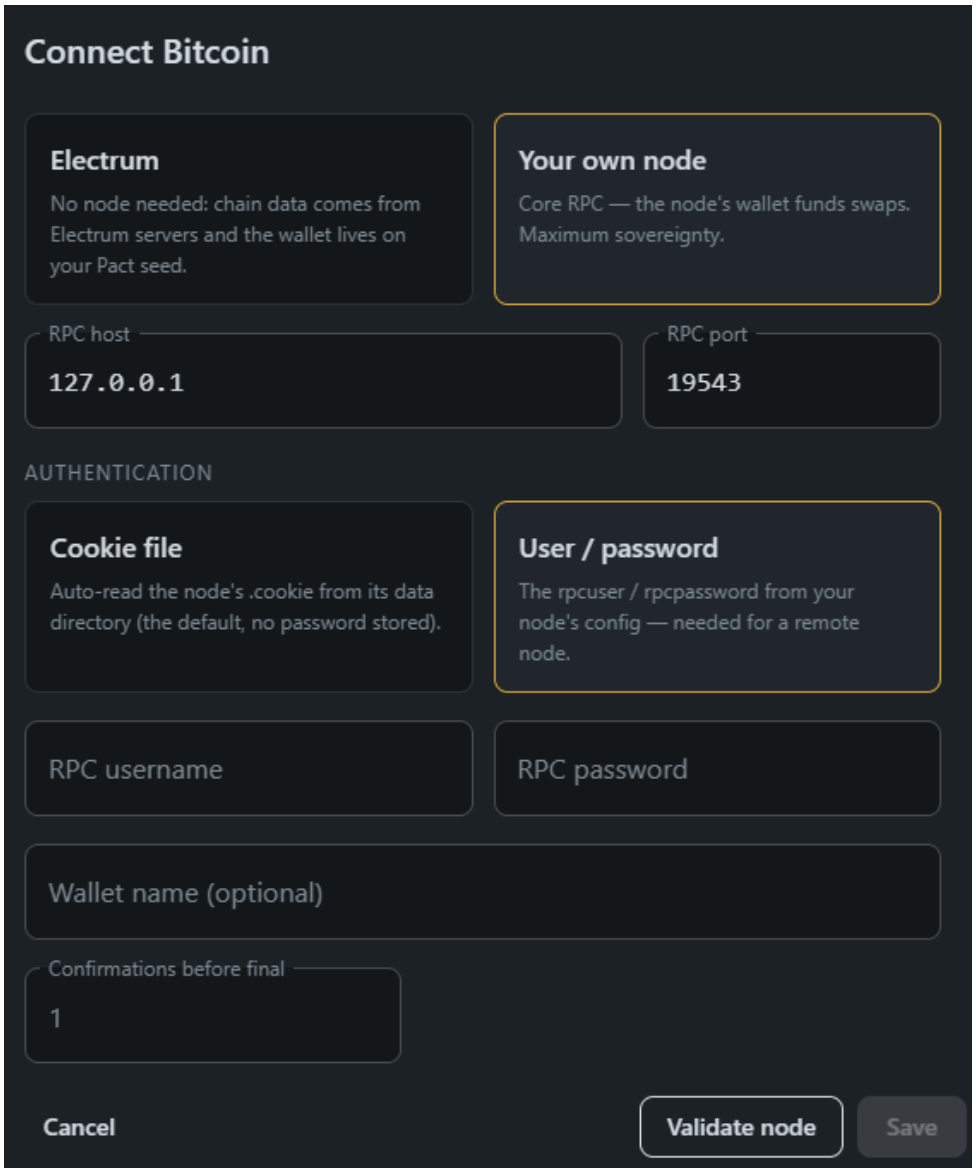

Figure 5.2: The coin connection form, set to read login from the node's cookie file.

Tip — On the same machine, the **Cookie file** option is the easiest and most secure choice: there's no password to type or store, and the login rotates automatically.

Reach for **User / password** mainly when connecting to a node elsewhere.

5.4 Validate before you save

Here is the part that keeps you safe from a costly mistake: **Satchel checks it's talking to the right blockchain before it saves anything.**

1. With the form filled in, click **Validate node**.
2. Satchel connects to the node and reads its *genesis block* — the unique first block that identifies a chain. It compares that to the genesis it expects for this coin and network.
3. You'll see one of:
   - **Checking the node…** while it works.
   - **Genesis matched — this is the right chain**, along with the node's current **tip height** and the genesis hash. Success.
   - **Rejected — not saving**, if the genesis doesn't match — meaning the node is on the wrong network (or it's the wrong coin entirely).
4. The **Save** button only becomes available **after** validation passes. If you change any field afterward, you'll need to validate again.

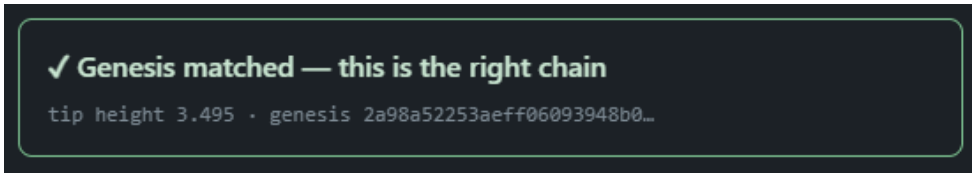

Figure 5.3: Validation succeeded: the node is on the correct chain, showing its tip height.

> **Warning** — This genesis check exists so your funds can **never** be sent into the wrong chain by a mis-typed port or a node running on the wrong network. Nothing is saved until the check passes — so if validation is rejected, **don't try to force it.** Fix the connection details (usually the port or the data directory) and validate again.

Click **Save**, and Satchel records the connection and reconnects the engine to that node. Repeat for your second coin.

5.5 Capabilities and trading pairs

Two more things appear on the Coins screen once a coin is connected.

Capability chips — small tags like **CLTV**, **SegWit**, and **Taproot** — describe technical features the coin supports. You don't need to understand them to trade; they simply tell Satchel which kinds of swap it can build with that coin. (If you're curious: *CLTV* enables the time-based safety refund, *SegWit* and *Taproot* are transaction formats — Taproot powers the newer, more private swap style.)

Trading pairs — below the coin cards, Satchel lists the pairs you can trade, derived automatically from what your connected coins can do. There's no fixed list. Each pair shows a readiness label:

- **Ready to trade** — both coins are connected and live. You're good to go.
- **Connect *(coin)*** — you still need to set up one of the two coins.
- **Not buildable yet** — the coins can't form a swap together (for example, one lacks a needed capability).

When **BTCX ↔ BTC** reads **Ready to trade**, you're set.

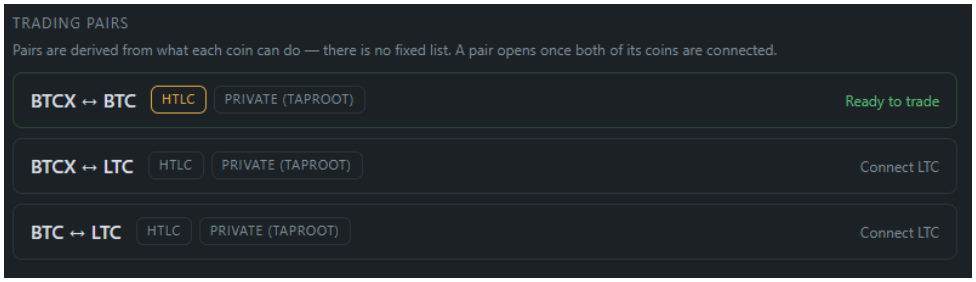

Figure 5.4: The Trading pairs list showing BTCX and BTC ready to trade.

5.6 Your credentials stay local

A reasonable worry: you've just typed in node details, maybe a password. Where does that go?

> **Note** — Your node credentials are **read and used locally, by the engine on your own machine — never shown back inside the app's window and never sent anywhere.** When you use a cookie file, Satchel reads it directly on your computer; the secret never even reaches the part of the app that draws the screen. Connections are shared across all your merchants, so you set each coin up once.

With two coins live and your pair reading **Ready to trade**, you're set: head to the **Corkboard**, ready for your first trade — which is exactly where Part 2 begins.

Part II

Trading

Chapter 6

A Tour of the Interface

Now that Satchel is set up, let's walk through the screen so you know where everything lives. Nothing here changes your funds — this is just an orientation tour. Open the app and follow along.

Satchel's window is split into three regions: a **left navigation rail** down the side, a thin **header** across the top, and the **main content area** that fills the rest. The chapters that follow each dig into one screen; this one shows you the map.

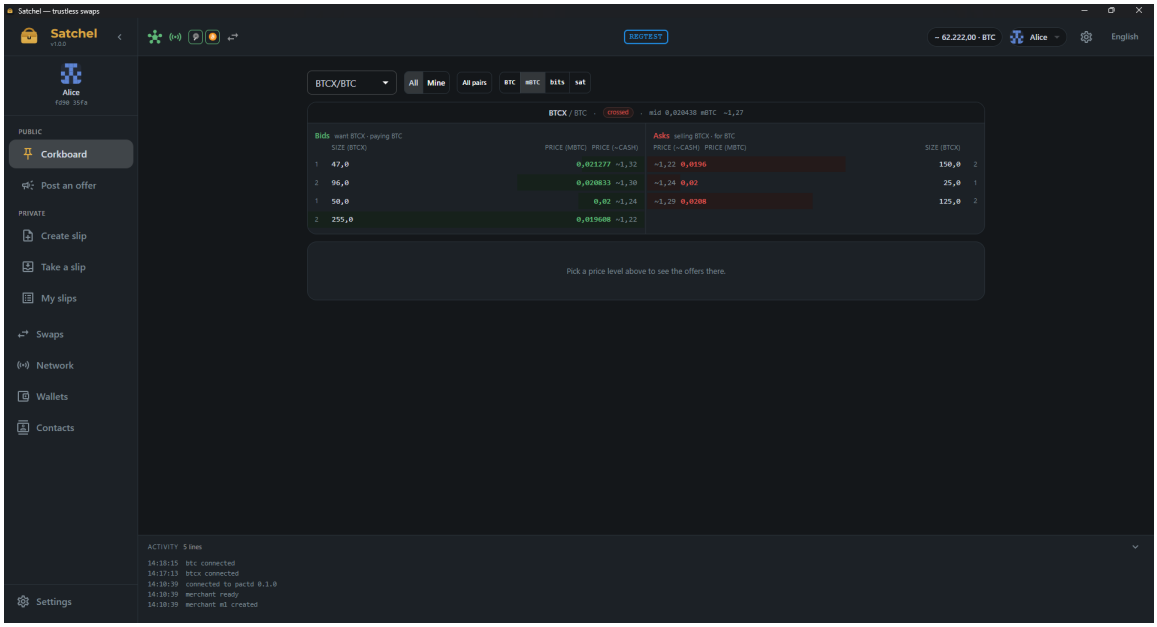

Figure 6.1: The Satchel main window: left navigation, header, and the Corkboard in the content area.

6.1 The left navigation

The rail on the left is how you move between screens. You can collapse it to a slim strip with the menu toggle in the header (handy on a small window), and on a narrow window it slides out as

an overlay. At the very top sits the **Satchel** logo with the version number underneath; if a newer release is available, a small update badge appears there — click it to see what's new.

The navigation items are grouped so related tasks sit together:

Public — trading out in the open, where anyone can see and take your offers:

- **Corkboard** — the order book. Browse every offer you can trade and take one. This is where the app opens.
- **Post an offer** — advertise a trade for anyone to take.

Private — trading with one specific person instead of the whole board:

- **Create slip** — build a private offer and get a shareable code (a *slip*).
- **Take a slip** — paste a slip a friend sent you.
- **My slips** — track and cancel the private offers you've handed out.

Below the groups:

- **Swaps** — your full trade history and the swaps currently running.
- **Wallets** — per-coin balances with **Send** and **Receive** (node-backed, or a pact-seed wallet over Electrum servers; Electrum coins add an **Activity** history).
- **Contacts** — your private nicknames and standings for the people you trade with, kept only on this device.
- **Network** — a read-only monitor of your connections: one tab for your Nostr relays and one tab per Electrum-backed coin, so you can see at a glance which relays and servers are connected and how healthy they are.

And at the foot of the rail:

- **Settings** — appearance, language, your coin connections, the noticeboards Satchel talks to, fee-bumping preferences, and desktop notifications. Its tabs are **General**, **Coins**, **Network**, **Fees**, **Notifications**, and **About**.

  Tip — If you ever feel lost, click **Corkboard**. It's the home base and it's where your live swaps and activity log are docked (more on that below).

The header also carries a **globe** menu — the language picker. Click it to switch Satchel's display language on the spot; it offers all **26 languages** Satchel ships with, listed under their own native names. It's always there, so you can change language any time, not just during first-run setup.

6.2 The active merchant, and switching between merchants

A *merchant* is one trading identity — its own recovery seed, its own swap history, kept separate from any other identity you create. Think of it like a single wallet profile. You set up your first merchant during onboarding.

Just under the logo you'll see a clickable **active-merchant block**: a little identicon, the merchant's name, and a shortened id. This always tells you which identity you're trading as. There's also a matching merchant chip on the right of the header.

You can **rename the active merchant in place**: hover the name in the block and click the little pencil (or click the name) to edit it inline — type a new label and press **Enter** to save, **Esc** to cancel. Only the label changes; the merchant's id, identity, and seed are untouched, so a rename is safe even with a swap in flight.

To switch, click either one. The merchant chip opens a dropdown with **Manage Merchants…** and a list of your other merchants. Choosing **Manage Merchants…** opens the full manager, where you can create a new merchant, import one from a recovery phrase, or load a different one. Creating runs the same short wizard as first launch — and the new merchant only comes into being at the wizard's final step, so backing out part-way changes nothing and your current merchant stays active.

> **Note** — Satchel won't let you switch away from a merchant that has a swap in flight. That's a safety rule: a running swap needs its own seed available to finish or refund. Wait for it to complete, then switch.

6.3 The Cashrate chip

Just to the **left of the merchant chip** sits the **Cashrate chip** — your own price reference. When it's on and you've set a rate it reads something like `~ 91,400.00 · BTC`; otherwise it just says **Cashrate (SYM)**. Click it and a small popover opens with an on/off toggle and a rate field: enter what *you* call one unit of the coin in your own money — EUR, USD, RMB, whatever you think in. Satchel never names a currency and never fetches a price; the rate is yours alone, remembered per coin.

The chip always binds to the **quote coin of the pair on the current screen** — the Corkboard pair, or the pair in an offer form. On screens with no coin context (Swaps, Wallets, Settings…) it greys out but keeps showing the last coin's rate. With a rate set, muted **~Cash** figures appear beside prices and amounts across the app; the Corkboard chapter covers exactly where and how to read them.

6.4 The header status indicators

The left of the header carries a row of small status lights. At a glance they tell you whether Satchel can do its job. Hover any of them for a plain-language tooltip.

- **Engine reachable** — a hub icon that is green when Satchel can talk to the engine (the core that holds your keys and runs swaps) and red when it can't. If this is red, nothing else will work until it recovers.
- **Nostr relay health** — a dot that only appears when you have relays configured. Green means at least one relay is connected; amber means none are reachable right now. (Relays are one way your offers travel — see *Settings* in the relevant chapter.)
- **Per-coin health** — a small glyph for each coin you've connected, bordered by its status. The tooltip tells you whether that coin's node is connected (and its current block height), not set up, or in error.

- **Live-swaps counter** — a gently pulsing swaps icon with a number badge showing how many swaps are running right now. Click it to jump straight to the Corkboard where you can act on them.

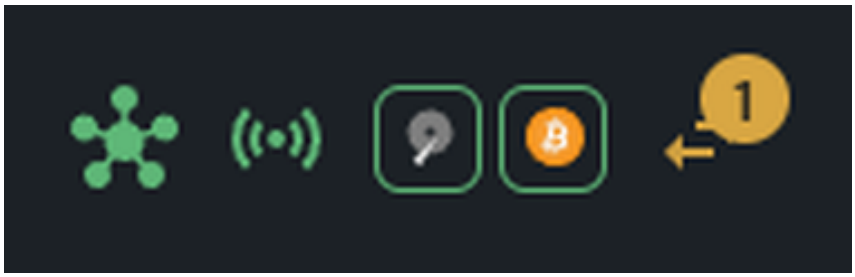

Figure 6.2: The header status indicators: engine, Nostr relays, per-coin health, and the live-swaps counter.

6.5 The network stamp

In the centre of the header you may see a network stamp such as **RegTest** or **TestNet**, reminding you that you are *not* using real funds. On **mainnet** (real coins) this stamp is deliberately hidden — there's nothing to warn you about, because that's the real thing.

> **Warning** — If there's no network stamp, you are on mainnet and trading with real money. Double-check amounts before you confirm anything.

If you haven't connected coins yet, you can still browse the board freely — nothing walls off the app. What's gated is the money-moving actions, and they're gated **per action** rather than app-wide: posting, taking, and funding stay out of reach until you connect two live coins, with each action pointing you to **Settings** → **Coins** at the moment you reach for it. (See *"Setting Up Your Coins"* for the details.)

6.6 The Network monitor

Click **Network** in the navigation to open a read-only health screen for your connections. It has one tab for your **Nostr relays** and one tab per **Electrum-backed coin**. It's a **monitor only** — it never dials or probes; every status you see comes from the connections your normal activity already uses.

6.6.1 Nostr relays

The relays tab lists one row per relay you've configured, so you can see whether your offers have a way out onto the network. The header reads **"{up} / {total} connected"** — how many of your relays are live right now — with a refresh button beside it. Each row then shows:

- A **status dot** — green when the relay is connected, amber while it's connecting, red if it was terminated or banned, grey otherwise.
- The **relay URL**.

- A short **status label**, the connection **latency** in milliseconds, and how long it's been **up**.

If you have no relays configured (or none are reachable), the screen points you to **Settings** to add some.

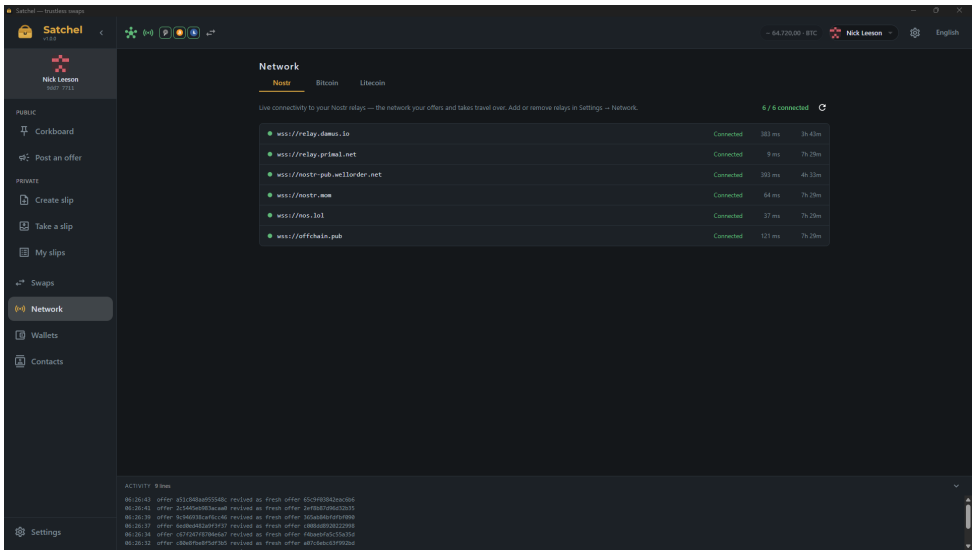

Figure 6.3: The Network monitor's Nostr tab: one row per relay with status, latency, and uptime.

6.6.2 Electrum servers (per coin)

Each coin that runs over Electrum servers gets its own tab, named for the coin. It lists every server you've configured for that coin, and shows how the app is using them right now:

- A **health dot** — green for healthy, red for a server in back-off (with a short retry countdown), grey for one that's simply never been needed yet.
- The **server URL** and its **role**: **wallet** (the one your balance and sends run through), **view** (an independent server the app cross-checks against), or **standby** (configured but idle, ready to step in).
- The current **state** and **latency**.

You don't manage anything here — the app picks a wallet server and a couple of views automatically, and if one drops it promotes a standby and moves on, so a single server going down never interrupts you. The **"{healthy} / {total}"** count in the header tells you how much headroom you have. It's the place to look if a coin ever shows a connection warning: as long as at least one server is healthy you're covered, and on mainnet a swap keeps two independent servers agreeing before it trusts anything on-chain.

> **Note** — This screen only *shows* you relay health; you don't add or remove relays here. To change which relays Satchel uses, go to **Settings** → **Network** (covered in the *Settings* chapter).

6.7 The activity log and active-swaps dock

Two panels are docked along the bottom of the content area, visible on every page:

- **Your active swaps** — a live card for each swap in progress, showing its state, the two parties as **maker** ↔ **taker** (your own side tagged **(you)**), the amounts, a live confirmation-progress line, and — while nothing of yours is locked yet — a **cancel** button to back out before funding. Funding, redeeming, and refunding all happen automatically, so there are no buttons for them; a **dump logs** button is always there for diagnostics. The chapter on tracking your swaps explains them.
- **Activity log** — a running, collapsible feed of what the engine is doing, narrated in plain language. You don't need to watch it, but it's reassuring to glance at when a swap is mid-flight.

6.8 The tray icon

Satchel also keeps a small icon in your **system tray**. Hover it and the tooltip mirrors the header's live-swaps counter — *"Satchel — no swaps in flight"*, *"Satchel — 1 swap in flight"*, and so on — so you can check on things without opening the window. A **left-click** brings the window back, and the icon's menu offers **Open Satchel** and **Quit**.

> **Note** — Quitting from the tray menu goes through exactly the same fund-safety exit gate as closing the window: if a swap is in flight, Satchel warns you and offers **Keep running** first. The tray can never bypass that protection. (If your desktop has no tray area, Satchel simply runs without the icon — nothing else changes.)

That's the whole map. The next chapters take each screen in turn, starting with the Corkboard.

Chapter 7

The Corkboard

The **Corkboard** is where trades live in public. Makers pin offers to it, and you browse those offers and take the one you like. It's the screen Satchel opens on, and it's the heart of the app.

One thing to understand up front: the Corkboard is a *noticeboard*, not an exchange. It never matches buyers with sellers, and it never holds your coins. It simply displays everyone's offers in a tidy, familiar layout so you can find the price you want and take it yourself.

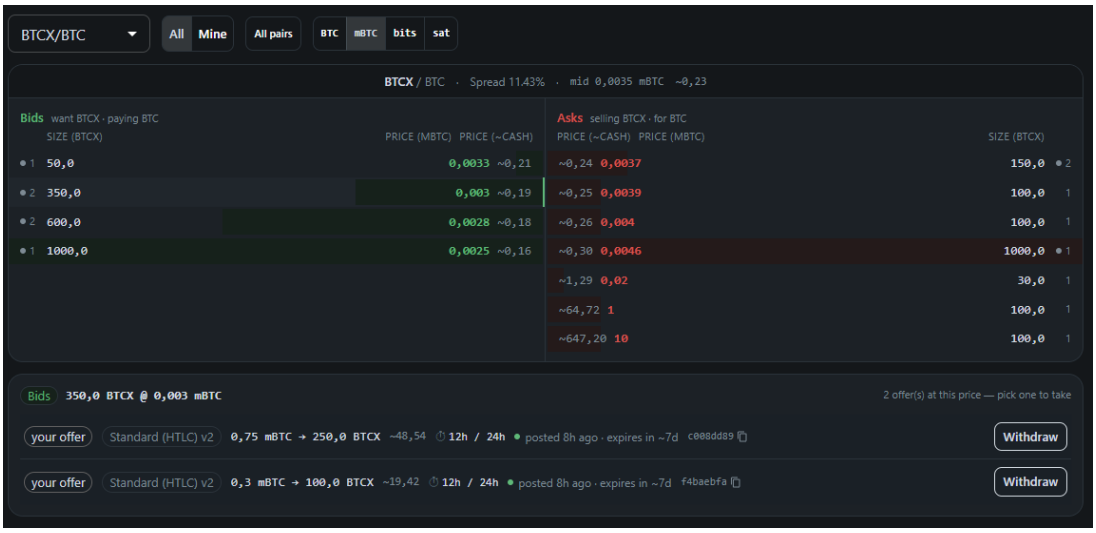

Figure 7.1: The Corkboard order-book ladder, BTCX/BTC.

7.1 The controls along the top

- **Pair selector** — chooses which two coins you're looking at, labelled **base/quote** (for example **BTCX/BTC**) in the same orientation as the order book it drives, so the selector and the book always read the same way round. Normally only pairs whose coins you've set up appear here — flip on **All pairs** (below) to browse the rest. Your last choice is remembered.
- **All / Mine** — **All** shows every offer on the board; **Mine** narrows it to just the offers you posted, so you can find and withdraw your own.

- **All pairs** — a toggle that widens the pair selector to **every pair actually on the board**, including coins you haven't set up, so you can window-shop the whole market. Offers on a pair you haven't connected are **view-only** — the **Take offer** button stays disabled until you set the coin up in **Settings** → **Coins**. Your choice is remembered. (The toggle appears once you have configured pairs to narrow the view against; with **no** coins set up the board already shows every pair, so there's nothing to widen.)
- **Denomination toggle** — switches the display unit for the quote coin, so you can read prices in whichever unit you think in.
- **Board selector** — if you've connected more than one noticeboard, this picks which one you're viewing. (You post to all of them at once, but you browse one at a time.)

7.2 Reading the order book

The book has two columns:

- **Bids** are on the **left** and shown in **green**. These are makers who want the base coin and are paying the quote coin for it.
- **Asks** are on the **right** and shown in **red**. These are makers selling the base coin for the quote coin.

Each row is a *price level*. Offers at the exact same rate are grouped together onto one level, so a single row can represent several offers. The **depth bar** behind the row shows how much coin is on offer at that price — a longer bar means more is available there. Within each column the best price sits closest to the centre divider: bids run high-to-low downward, asks run low-to-high.

To keep things readable, Satchel shows the top eight levels per side. If there's more, a **Show more** toggle reveals the rest.

7.2.1 The spread banner

Above the two columns is a small banner describing the gap between the best bid and the best ask:

- A **spread percentage** and a **mid price** when there are offers on both sides (with the mid's **~Cash** equivalent beside it, if you've set a Cashrate — see below).
- **One-sided** when there are offers on only one side.
- A **crossed** chip when the top bid is at or above the top ask.

That last one can look alarming if you're used to an exchange, so here's the key point: **the board never matches trades**. When offers overlap, they simply sit there — nothing fires automatically. You can take either side at the price shown. Crossed just means two makers happen to have posted overlapping prices.

> **Note** — On an exchange, a crossed book would instantly execute. On the Corkboard nothing executes on its own — *you* choose an offer and take it. The ladder is only a way to *read* the board.

7.3 Prices in your own money: the Cashrate

Coin-per-coin prices are exact, but they're not how most people *think*. The **Cashrate** lets you read the board in your own money — without Satchel ever touching a price feed.

Click the **Cashrate chip** in the header (just left of the merchant chip) and a popover opens with an on/off toggle and a rate field. Enter what you call one unit of the **quote coin** in whatever money you think in — EUR, USD, RMB, anything. The chip always binds to the quote coin of the pair you're looking at, and each coin's rate is **remembered separately**, so switching from BTCX/BTC to BTCX/LTC recalls the LTC rate you set last time. On screens with no coin context (Swaps, Wallets, Settings…) the chip greys out until you're back on the board or an offer form.

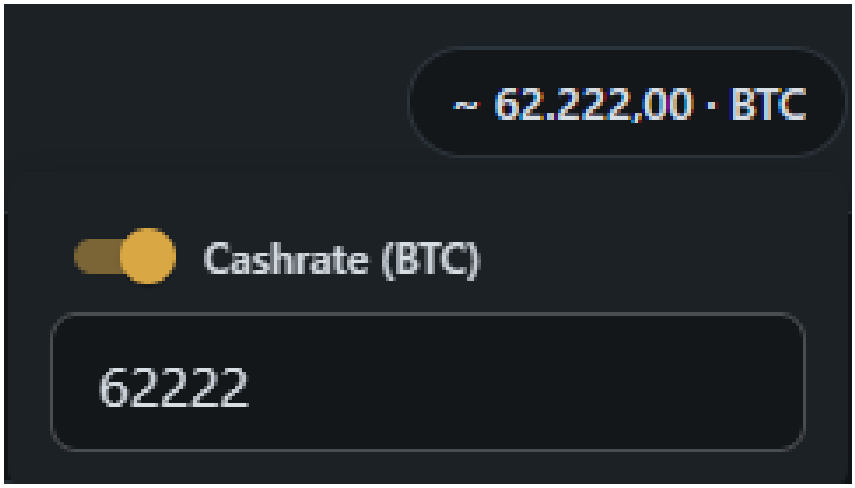

Figure 7.2: The Cashrate chip and its popover: the on/off toggle and the rate field for the current quote coin.

Once a rate is set, muted **~Cash** figures appear beside the real numbers:

- **In the ladder** — an extra unit-price column on each side, hugging the centre divider, showing each price level in your money. (The columns disappear entirely when the Cashrate is off.)
- **In the spread banner** — the cash equivalent next to the mid price.
- **On offer rows** — one figure per offer next to the amounts. An offer's two legs are worth the same at its own price, so a single figure values the whole trade.
- **In the offer form and the take-offer confirmation** — beside the "You give" / "You get" lines, so you sanity-check a trade in familiar terms (see the next two chapters).

A few things worth knowing about how ~Cash is designed:

- **It's deliberately currency-neutral.** Figures render as ~ plus a bare number, always with two decimals — never a $ or a currency name. The money is whatever *you* meant when you typed the rate.
- **It's display-only.** Satchel never fetches a price and makes no external calls for this. That's by design, not a gap: it keeps your browsing private, and BTCX isn't listed anywhere a feed could price it anyway. Your rate is your reference, not a market price.
- **It's off by default.** Until you flip the toggle on, no ~Cash figure appears anywhere.

Tip — Treat ~Cash as a sanity check, not a quote. It tells you what a trade is worth *at the rate you believe in* — a quick way to spot an offer that's priced far from your own view of the market.

7.4 The detail pane: seeing the offers at a level

Clicking a price level fills the **detail pane** below the book with the individual offers sitting at that rate, biggest first. (Before you pick a level, the pane prompts you to "pick a price level above.")

Each offer row shows:

- **The counterparty** — a tag for who posted it, or "your offer" if it's yours. An offer you've *just* posted shows a dimmed, italic **"posting…"** badge (and a hollow dot in the ladder) for the brief moment between when you post it and when a relay echoes it back; once confirmed it switches to the normal "your offer" tag and goes solid. See *Posting an offer* below.
- **Amounts from your perspective** — what you would give and what you would receive if you took it. You don't have to do the maker's math in reverse; Satchel flips it for you. (With a Cashrate set, a muted **~Cash** figure beside the amounts values the trade in your own money.)
- **The protocol chip** — either **Standard (HTLC)** (the classic swap, muted style) or **Private (Taproot)** (the newer private swap, highlighted style). Both are safe; the private one simply looks like an ordinary payment on-chain. See the safety chapter for the difference.
- **The safety-refund times** — shown as `t2h / t1h` (for example `12h / 24h`). These are the auto-refund deadlines that protect you if a swap stalls; the taker's side unlocks first, the maker's a little later, so nobody gets stuck.
- **A freshness dot** — how recently the offer was seen, so you can favour live ones.
- **A copyable offer id** — a short, muted id with a copy button, so you can reference a specific offer when chatting with a counterparty.

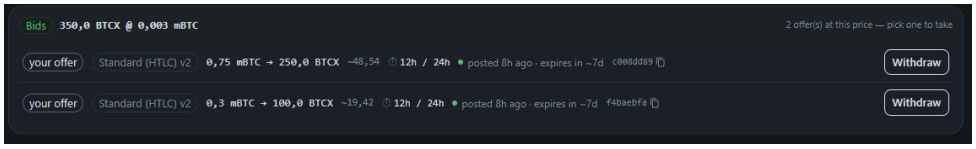

Figure 7.3: The detail pane: individual offers at one price level, with protocol chips and refund times.

7.5 Knowing the maker: contacts on the board

If you've saved a maker as a contact, the board recognises them: their **nickname** and a small **status dot** for their standing show on the offer's counterparty tag, so a familiar trader stands out from a stranger at a glance. And once you've blocked anyone, a **Hide blocked offers** toggle appears above the board — turn it on to drop blocked makers' offers off the ladder entirely. See the chapter *"Contacts"* for how to add a maker and what standing does (and, importantly,

doesn't) do.

7.6 Taking an offer

When you've found an offer you like, click **Take offer** on its row. Satchel shows a confirmation dialog summarising the trade — amounts, protocol, refund times, and the network-fee preview — before anything happens. The chapter on taking an offer walks through what comes next.

The **Take offer** button is disabled in two cases:

- **It's your own offer** — you can't take yourself. (Use **Withdraw** instead, see below.)
- **A node is down** — if one of the two coins for this pair isn't reachable, Satchel blocks the take with a note to start the node or check **Settings** → **Coins**. This is a friendly guard; the engine would refuse it anyway.

7.7 Your own offers on the board

When you post an offer, it shows up on the board **instantly** — you don't wait for the network. While it's still travelling out to a relay it wears a dimmed, italic **"posting…"** badge (a hollow dot in the ladder), which simply means *"posted from this device and already advertising; waiting to be confirmed back from a relay."* The moment a relay echoes it, the badge becomes the ordinary **"your offer"** tag and the dot fills in — it's now fully live.

> **Note — Closing takes your offers down; restarting offers them back.** When you close Satchel it stops advertising your offers, so the board never shows listings you can't honour while you're away. Nothing is re-posted behind your back: the next time you start Satchel, a **"Bring back your offers?"** dialog lists every offer that was still within its valid-for window. **Revive** re-posts one at the same terms as a brand-new offer (a fresh listing with a full validity window — prices may have moved while you were away, which is exactly why you get the choice); **Dismiss** keeps it withdrawn; closing the dialog dismisses whatever's left. An offer whose coin you've since removed can't come back and is dismissed automatically. Offers past their valid-for window simply expire. And as ever, an explicit **withdraw** removes an offer for good — it never comes back, revived or otherwise.

7.8 Withdrawing your own offer

For an offer you posted, the row shows **Withdraw** instead of **Take offer**. Click it to pull the listing immediately. Because an offer never locks any funds, withdrawing is instant and costs nothing — it just removes the advert.

7.9 Empty and error states

The Corkboard tells you clearly when there's nothing to show:

- **No Corkboard connected** — you haven't pointed Satchel at a noticeboard yet. A **Configure in Settings** link takes you there.
- **No offers you can take right now** — the board is reachable but has no offers for a pair you've set up. They'll appear as soon as a maker posts one, and you can always post your own.

Out of the box, the Corkboard only shows offers for pairs whose coins you've connected — offers for other pairs simply don't appear. To *see* them anyway, flip on the **All pairs** toggle in the toolbar and browse the whole board; to *trade* a pair, you still need both of its coins set up in **Settings** → **Coins**.

Chapter 8

Making an Offer

Instead of taking someone else's offer, you can post your own and let others come to you. Click **Post an offer** in the left navigation to open the offer form.

Posting an offer locks nothing. It's just a signed advert pinned to the board. A swap only starts when someone takes your offer and both sides fund their legs — and you can withdraw the offer any time before that. So there's no risk in posting; take your time getting the terms right.

8.1 Filling in the form

The form reads top to bottom the way you'd describe a trade out loud: *which pair, which way, how much, at what price*.

8.1.1 Pair

First choose the **Pair** you want to trade — a single canonical pairing such as **BTCX / BTC**. The dropdown lists every pair your connected coins can form, and it defaults to whatever pair you were last looking at on the Corkboard. If it's empty, you simply haven't connected enough coins yet — the form tells you to connect at least two under **Settings** → **Coins**.

8.1.2 Sell / Buy

A two-button **direction toggle** sets which way you're trading the pair's *base* coin: **Sell {base}** means you give the base coin, **Buy {base}** means you receive it. (If you've used an earlier version: the old flip button is gone — this toggle replaces it, and it's clearer about what's happening.)

8.1.3 Amount

The **Amount** is always entered in the **base coin**, with the base symbol shown at the end of the field. Just beneath it Satchel shows your **live balance** in that coin (pulled straight from your own node), so you can see what you have to work with.

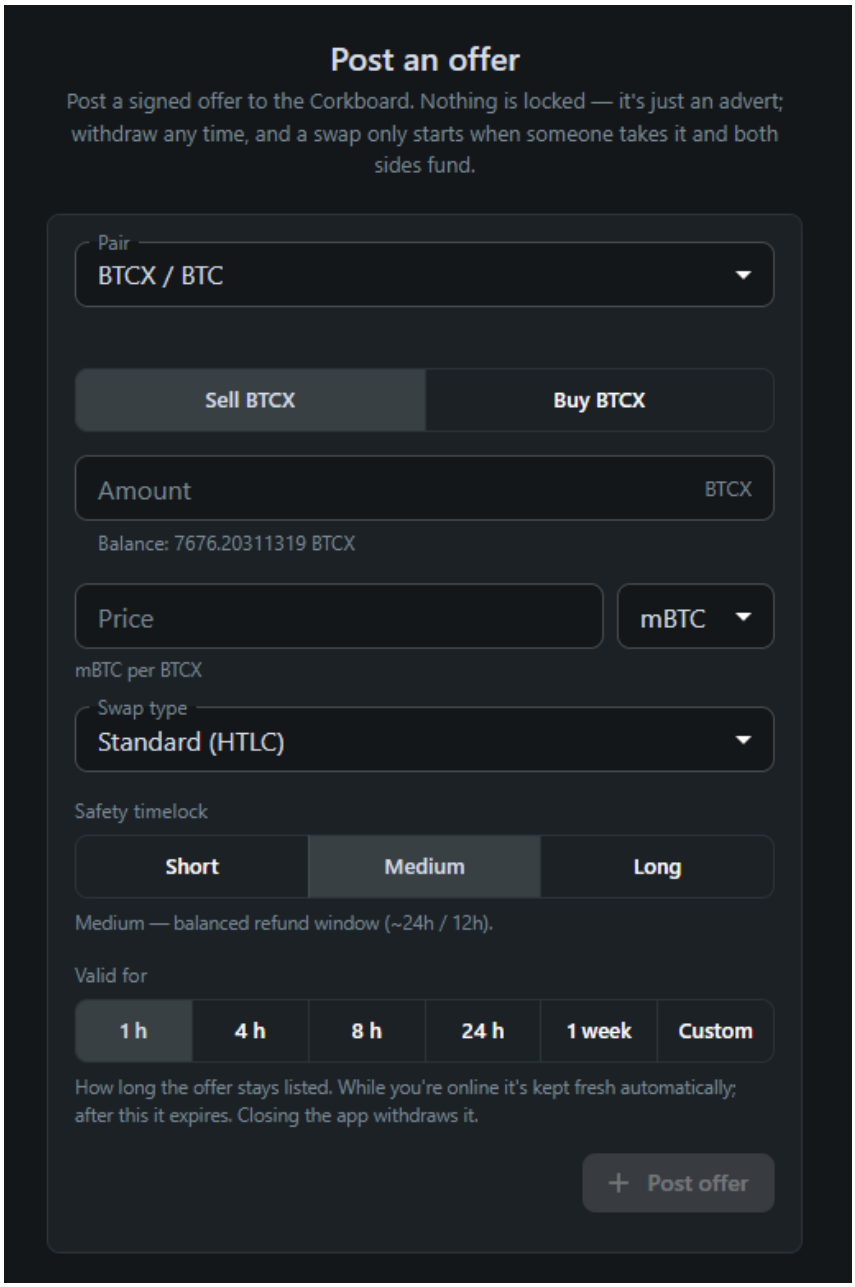

Figure 8.1: The Post an offer form: pair, direction, amount, price, swap type, and safety timelock.

There's also a floor. An offer where **either side of the trade comes to less than about 3,430 sats** is refused up front, with a message saying the leg is too small. That isn't fussiness: a swap that tiny could lock funds and then be worth less than the network fee needed to claim or refund it near the deadline — stranding coins that can never economically move again. Refusing it before anything is posted means a swap can never start that couldn't safely finish. (The same floor applies when someone tries to *take* an offer.)

8.1.4 Price

The **Price** is always quoted **quote-coin per base-coin**, and — this is the nice part — it **doesn't change when you flip Sell to Buy.** The price of the pair is the price of the pair; only the direction of the trade changes. A small label spells out the unit ("{unit} per {base}") and shows the raw rate as a hint — and if you've set a **Cashrate** (the header chip — see the Corkboard chapter), the hint also appends the unit price in your own money, like `· ~91,400.00 / BTC`.

Next to the price is a **denomination dropdown** — **BTC / mBTC / µBTC / sat** — so you can quote in whatever unit you think in. This is the **same** denomination setting the Corkboard uses, so the two always agree and never drift apart.

8.1.5 Give / get summary

From your amount and price, Satchel shows a plain **"You give …" / "You get …"** summary so there's no ambiguity about what you're actually offering before you go any further. With a Cashrate set, each line also carries its muted **~Cash** equivalent — a quick sanity check that the trade is worth what you think it is, in your own money.

8.1.6 Swap type

If the pair you chose supports more than one kind of swap, a **Swap type** dropdown appears so you can pick:

- **Standard (HTLC)** — the classic atomic swap.
- **Private (Taproot)** — a newer style that looks like an ordinary payment on-chain.

If the pair only supports one type, Satchel just shows it as a line of text — there's nothing to choose. Both are equally safe; the safety chapter explains the difference. **Private (Taproot)** is now selectable on **mainnet** too, not just on the test networks, whenever both coins support Taproot.

8.1.7 Safety timelock

The **Safety timelock** is the auto-refund window: if a swap stalls, this is how long before your funds automatically come back to you. Rather than ask you for raw block times, Satchel offers three presets:

Preset	Refund times (t2 / t1)	What it means
Short	~6h / ~12h	Funds auto-refund fastest if a trade stalls — but the smallest safety margin.
Medium (default)	~12h / ~24h	A balanced window. Recommended for most trades.
Long	~18h / ~36h	The widest safety margin; auto-refund takes longest if a trade stalls.

The two numbers are the two sides' refund deadlines (the taker's leg unlocks at the shorter one, yours at the longer). A **shorter** timelock gets your money back faster if something goes wrong, but leaves a thinner margin for the swap to complete normally. A **longer** one is the safest choice but means a stalled swap takes longer to unwind. When in doubt, leave it on **Medium**.

> **Tip** — The timelock only matters if a swap *stalls*. A normal swap completes in minutes; the timelock is purely the safety net.

8.1.8 Valid for (minutes)

Valid for sets how long the offer stays listed, in minutes (default 60). While Satchel is open, the engine keeps the listing fresh automatically; once the window passes, the offer expires. Closing the app takes it off the board too — on the next start Satchel asks whether to revive it as a fresh listing or keep it withdrawn (see the chapter *"The Corkboard"*).

8.2 Reviewing and confirming

When the form is complete, click **Post offer**. Satchel shows a **review dialog** before anything goes out, summarising:

- what you give and what you receive,
- the swap type,
- the safety-refund window (`t2h / t1h`),
- how long the offer is valid, and
- a **network-cost preview** so you know roughly what the on-chain fees will be.

Check it over, then confirm.

8.2.1 The funds check

Before it lets you confirm, the review dialog quietly checks that your wallet can actually fund your side — and it checks for the **amount plus the on-chain funding fee**, not just the bare amount. If you'd come up short, it shows a red **"Not enough … (amount + funding fee)"** alert

telling you roughly what you need versus what you have, and the confirm button stays **disabled** until you lower the amount or top up. This catches the awkward case where you have *almost* enough but not quite enough to also pay the miner.

> **Tip** — Fee figures in the preview are always shown to **8 decimal places** (for example `0.00100000`), so you can read them exactly without rounding surprises.

> **Note** — There are **no platform fees**. The Corkboard charges nothing to post or trade. The only cost is the ordinary on-chain mining fee you'd pay for any Bitcoin-style transaction, and that's only paid once a swap actually runs.

8.2.2 The wallet-lock check

Satchel also checks, before it posts, that the node wallet for the coin you're **giving** isn't locked. If that wallet is encrypted and currently locked, posting is refused up front with a clear message telling you to **unlock it first** — run `walletpassphrase` on the node — and to keep it unlocked until the swap completes.

> **Warning** — A locked wallet can still *read* its balance but cannot *sign* the funding transaction. Without this check the offer could be taken and then strand at funding, unable to lock your side. Unlock the give-coin's wallet before you post, and leave it unlocked for the life of the swap.

8.3 What happens after you post

Your offer **fans out to all of your noticeboards at once** — every Corkboard and every Nostr relay you've configured — so the widest possible audience can see it. Satchel then takes you to the Corkboard, where your own offer is marked with a "your offer" tag and a "mine here" dot at its price level.

From there you simply wait. If someone takes your offer, a swap begins and you'll see it in **Your active swaps** and on the **Swaps** page. If you change your mind first, find the offer (the **Mine** filter helps) and click **Withdraw** — it's instant and free.

Chapter 9

Taking an Offer & How a Swap Runs

This chapter follows what happens from the moment you take an offer to the moment the swap completes. The good news is short: once you confirm, the engine does the work. You don't have to babysit it.

9.1 Taking an offer from the Corkboard

On the Corkboard, open the price level you want, find the offer, and click **Take offer**. Satchel shows a **confirmation dialog** before committing anything. It lays out:

- **You give** and **You receive** — the exact amounts, from your side. If you've set a **Cashrate** (the header chip — see the Corkboard chapter), both rows also show their muted **~Cash** equivalents in your own money.
- **Counterparty** — who you'd be trading with.
- **The protocol** — **Standard (HTLC)** or **Private (Taproot)**.
- **Safety refund** — the auto-refund deadlines (`t2h / t1h`).
- **A network-cost preview** — the on-chain fees you'll pay.

Crucially, it also reassures you about order of operations: *the maker locks their coins first — you never send first*. You can still cancel before you fund your own side, and if anything stalls the engine auto-refunds you after the safety timelock.

Just like posting an offer, the dialog runs a **funds check** first: it confirms your wallet can cover the **amount plus the funding fee** for your side, and if it can't, it shows a "Not enough … (amount + funding fee)" alert and blocks the take until you can.

It also runs the same **wallet-lock check** posting does, on the coin you'd be **getting** (the side you fund): if that node wallet is encrypted and locked, the take is refused up front, asking you to unlock it first with `walletpassphrase`. And the same **minimum size** rule from posting applies here too: a take is refused if either side of the trade comes to less than about 3,430 sats — too small to claim or refund safely once network fees are paid.

> **Warning** — A locked wallet can read its balance but can't sign your funding transaction, so taking with it locked would strand the swap at funding. Unlock the get-coin's

wallet before you take, and keep it unlocked until the swap completes.

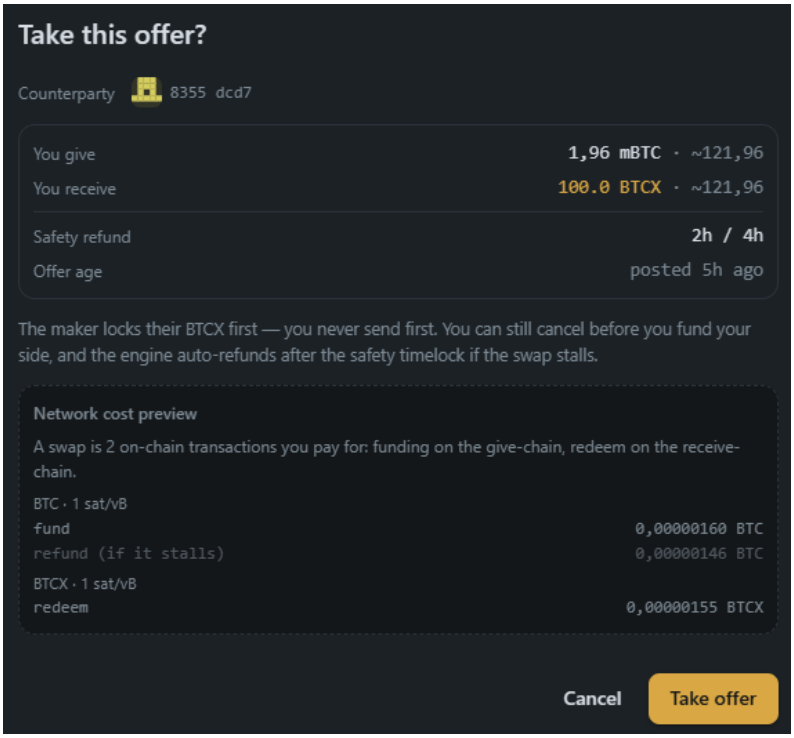

Figure 9.1: The take-offer confirmation dialog.

If the offer was posted by someone you've marked **Blocked** in your contacts, the dialog also shows a warning — **"You blocked this counterparty"** — and asks you to confirm a second time. This does **not** hard-block the trade: blocking is only a personal reminder, and an atomic swap protects you regardless of who you trade with. See the chapter *"Contacts"*.

Read it over and confirm. The swap is now under way.

9.2 How the swap runs, in plain language

A swap is an *atomic* trade: either both sides happen, or neither does. There's no moment where one person has both coins. Here's the sequence, without the cryptography:

1. **The maker funds first.** The party who posted the offer locks their coins on their chain before you lock anything. You are never the one exposed first.
2. **Your side is funded automatically.** When the swap reaches the point where it needs *your* coins, Satchel funds your side for you — you don't have to click anything. This is how every swap works — there is nothing to configure.
3. **The engine watches both chains.** Satchel's engine monitors the blockchains for both coins, waiting for each funding to confirm to a safe depth.
4. **The funds are released atomically.** Once both sides are locked, the engine completes the exchange so that each of you can only claim your coin in a way that simultaneously lets the other claim theirs. Neither side can run off with both.
5. **If anything stalls, you're auto-refunded.** If a counterparty disappears or a chain gets

stuck, the engine waits for the safety timelock and then automatically pulls your locked funds back to you. You don't have to do anything to get a refund — it's built in.

Note — The whole flow is automatic: after you take an offer you won't touch a button at all — funding, redeeming, and (if needed) refunding all happen on their own. The only manual control in the active-swaps dock is **cancel**, to back out *before* you've funded; once your funds are committed the engine sees the swap through — see the chapter on tracking your swaps.

9.3 You don't have to babysit it — but keep the app running

You don't need to sit and watch a swap. But the **engine must keep running** until the swap finishes, because it's the engine that redeems or refunds at the right moment. Those moments are governed by on-chain timelocks with real deadlines.

If you try to close Satchel while a swap is live, it offers to **Keep running** — this leaves the engine working quietly in the background (even with the window closed) so the swap finishes on its own. This is the recommended choice. You can reopen Satchel any time to check progress.

Warning — Force-quitting in the middle of a swap stops the engine and can lose funds, because nothing is left running to redeem or refund before the deadline. Always choose **Keep running** when prompted. The exit dialog is covered in the chapter on tracking your swaps.

9.4 Where to go next

- To watch a swap progress and act on it if needed, see the chapter **Tracking Your Swaps**.
- For the reassurance of *why* this is safe — what atomic means, what the timelocks guarantee — see the safety chapter.

If you're curious about the cryptographic machinery (HTLCs, adaptor signatures, the actual scripts), that's all in the companion **Pact** handbook. You don't need any of it to trade safely.

Chapter 10

Tracking Your Swaps

Once a swap is under way, you can follow it from start to finish. There are two places to look: the **Swaps** page, which is your read-only ledger, and the **active-swaps dock**, which appears beneath every page and is where the occasional action buttons live.

> **Tip** — You don't have to keep the window in front, either. When Satchel is in the background, swap milestones also arrive as **desktop notifications** (configurable under **Settings** → **Notifications**), and the **tray icon**'s tooltip always shows how many swaps are in flight. While the window has focus, notifications stay silent — the dock is already telling you the story.

10.1 The Swaps page

Click **Swaps** in the left navigation. This page is a **read-only ledger** — it shows you everything but has no action buttons (those live in the active-swaps dock, covered below). It's split into two sections, newest first:

- **In flight** — swaps currently running.
- **History** — swaps that have finished, refunded, or been aborted.

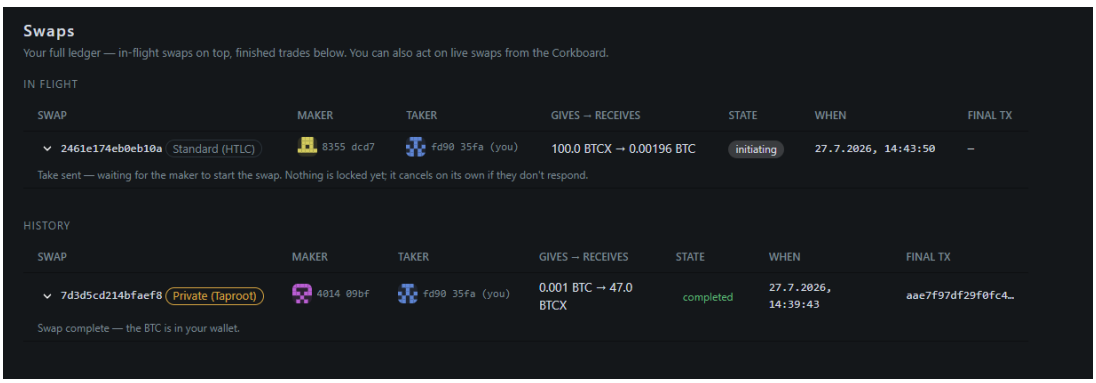

Figure 10.1: The Swaps page: in-flight swaps above, finished trades below.

Note — The ledger lists **this machine's** swaps. If you run the same recovery phrase

on more than one machine (see the *Backup, Seeds & Safety* chapter), the other machine's swaps appear read-only in the active-swaps dock under **Another machine** — they join this ledger only once you take them over.

10.1.1 The columns

Each row shows:

Column	What it tells you
swap	The swap's short id.
maker	The party who posted the offer — shown as a small identicon and short public key.
taker	The party who took it — likewise an identicon and short key.
gives → receives	What you gave and what you got.
state	Where the swap is in its lifecycle.
when	The timestamp.
final tx	The settling transaction once it completes.

Both parties are shown explicitly, side by side, rather than as a single "your role" label: the **maker** is whoever posted the offer and the **taker** is whoever took it, regardless of which one is you. Your own side is marked with a **(you)** tag next to its key, so you can always tell which party you are at a glance. A party Satchel hasn't recorded yet (occasionally the case for an older record) shows as **unknown**.

Every swap carries a **protocol chip** so you can tell the two kinds apart at a glance: a muted **Standard (HTLC)** chip for a classic swap, and a highlighted **Private (Taproot)** chip for a private one. The same chip appears on each card in the active-swaps dock.

10.1.2 Reading the state

The **state** is colour-coded so you can scan the page quickly:

- **Finalizing** — your claim has been broadcast and you're just waiting for it to bury under enough confirmations to be safe. The coins are effectively yours, but the engine still needs to run, so the swap stays **active** (it keeps a card in the dock, counts toward your in-flight total, and the exit gate still warns you). Keep the app open until it finishes.
- **Completed** — green. The swap finished, your claim is **buried and safe**, and the engine is done — you can close the app freely. ("Completed" deliberately appears only once the claim has confirmed deep enough, not the instant it's broadcast.)
- **Refunded** — amber. The swap didn't complete, so your locked funds came back to you. No loss.
- **Aborted** — red. The swap was cancelled before any funds were locked.

Alongside the state, Satchel shows the engine's own narration — a plain-language, verbatim description of what happened at each step. This is the same calm, running commentary you see in the activity log. It marks the turning points for you: the moment your lock transaction goes out, the story says so — and notes that from here the timelocks, not cancelling, are your safety net.

10.1.3 The live progress line

While a swap is in flight, a compact **progress line** sits just beneath the narration — both here and on each card in the active-swaps dock. It turns the qualitative story into a number you can watch tick up, and it shows only while there's genuinely something to wait on (it disappears the moment a step is final, so it never sits "stuck" once a leg has buried). Depending on whose move you're waiting for, you'll see one of:

- **Awaiting their lock** / **Awaiting their claim** — you've done your part and are waiting on the counterparty. There's no fixed target to count toward, so the bar is indeterminate and a small **+N blocks** shows how many blocks have passed while you wait.
- **Locking your {coin} — unlock wallet if stalled** — *your own* funding of a leg is pending or retrying: it hasn't broadcast or confirmed yet. The bar is indeterminate, with a **+N blocks** liveness count beside it; a count that keeps growing flags a fund that's genuinely stuck, most often because the give-coin's node wallet is locked. This state is honest about *whose* action is outstanding — it's yours. (Previously a stuck taker in this position mis-displayed as though it were the maker's turn; now it names the wait correctly.)
- **Your lock confirming · 2/6** — *your own* funding is burying toward the depth your counterparty needs before they'll lock their side. You'll see this as the maker of either kind of swap: the wait is on your lock maturing, not on a slow counterparty, so Satchel shows it honestly as a count on your own chain rather than an indeterminate "awaiting their lock." (A Private swap's maker locks the moment its offer is taken, so this count starts during the brief signing handshake and runs until your lock is deep enough — only when the taker's lock is actually *seen* on the network does the line switch to **Their lock confirming**, never before.)
- **Their lock confirming · 3/6** — their funding is burying toward the depth you need before you act. The numbers are *confirmations so far / confirmations needed*. Each side picks its own confirmation depth in coin setup, and the two of you exchange those choices at the start of the swap purely for display — so the target in a "your lock" count is the depth *they* chose to wait for, and the target in a "their lock" count is *yours*. The exchange only makes the counters precise; what each side actually waits for is always its own setting.
- **Securing your BTC · 2/6** — your own claim — a redeem *or* a refund — is burying toward final. When it reaches the needed depth the swap flips to **Completed** (or **Refunded**).

Where it's relevant, the line also shows the current settlement **feerate** (for example `· 2 sat/vB`), so any automatic fee-bumping is visible to you rather than hidden. It's purely informational — the engine drives the swap either way.

> **Note** — If a funding transaction fails to broadcast — for instance because the node

wallet got locked partway through a swap — the engine **re-attempts it automatically on every tick**. So if you see **"Locking your {coin}"** sitting stuck, just unlock the wallet (`walletpassphrase`) and the swap **self-heals** on the next pass, with no manual step from you. The retry is idempotent — it locates any funding already on chain first, so it never double-funds — and this locate-first guard now covers **both** Standard (HTLC) and Private (Taproot) swaps. A **Standard** swap also auto-retries its funding on every tick. A **Private (Taproot)** swap instead broadcasts *your* leg only once both sides have signed **and** the counterparty's leg is confirmed on-chain — so you never lock first-in-the-dark — and a Taproot swap that funds and then stalls auto-refunds at its timelock, so nothing strands unattended either way.

10.1.4 On-chain detail

Each swap row can be expanded to reveal its **on-chain detail** — the audit trail. This shows both funding transactions and *your* settlement or refund (never the counterparty's settlement, and never the swap secret). Each transaction id has a **copy** button, so you can paste it into a block explorer if you ever want to see it on-chain yourself. The expanded row also carries a **Dump logs** button for support — see *"Dump logs: diagnostics for support"* below.

> **Tip** — You never *need* to check a block explorer; the swap completes on its own. The on-chain detail is there for peace of mind and your own records.

If you have no swaps yet, the page simply says so and points you to the Corkboard to take your first offer.

10.2 Where the actions live: the active-swaps dock

The Swaps page shows you everything but doesn't have buttons. The handful of actions a swap might ask of you live in the **active-swaps dock**, which is docked beneath every page. Each live swap gets a card there with its state, the two parties shown as **maker** ↔ **taker** (each an identicon and short key, your own side tagged **(you)**; hover the arrow to see which side is which), the amounts, the engine's narration, and a "refund *when*" note.

The dock keeps just one action button, plus diagnostics:

Button	When it appears	What it does
cancel	While nothing of yours is locked yet (no funding on either leg)	Abandons the swap; you lose nothing, since nothing of yours is locked.
dump logs	Always	Copies a secret-free diagnostics bundle for support (see below).

There is no **redeem**, **refund**, or **fund** button. Funding, redeeming, and refunding are all au-

tomatic — the engine funds your side as the trade begins, auto-redeems the instant it's safe, and auto-refunds anything past its timelock. Every one of those steps is also **chain-gated** (by confirmations and timelocks), so a manual button could never make them happen sooner or differently — only fail or double-act. The one genuinely human decision is backing *out* before any funds are committed, which is what **cancel** is for. It shows a confirmation dialog first.

> **Note** — **cancel** is offered only while nothing of yours is locked yet, so it's always safe — the offer simply won't complete. It's gated on there being no funding on either leg (not on a particular state name), so it behaves correctly for both **Standard (HTLC)** and **Private (Taproot)** swaps, and it works even on an "initiating" pre-swap that's still waiting on the maker to answer. Once a leg is funded, your safety net is the **"refunds at *when*"** time shown on the card: the engine reclaims your funds automatically after that deadline.

> **Tip** — You don't strictly have to press **cancel** on a handshake that's going nowhere. A swap of either kind stuck waiting on the other side — before anything is funded — clears itself automatically after about 15 minutes, on both sides independently, with nothing lost either way. Cancel is there for when you don't want to wait even that long.

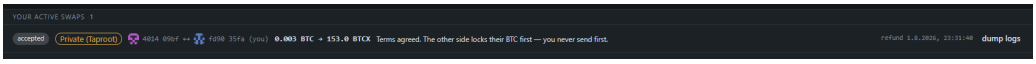

Figure 10.2: The active-swaps dock, with its cancel and dump-logs buttons.

10.2.1 Swaps from another machine

If you run the same recovery phrase on more than one machine (see *"One seed on more than one machine"* in the *Backup, Seeds & Safety* chapter), the dock also shows the *other* machine's in-flight swaps, grouped per machine under a heading like **Another machine · M-7f3a**. Those cards are **read-only**: this machine watches them on-chain but never acts on them, and they don't appear in the Swaps ledger. The narration keeps the roles straight — when the next move is your own side's lock, the card says so plainly: *your side's lock is next — the machine driving this swap sends it*, so you're never tempted to act from here. And the watching is careful, even on a coin reached only through your own node: a swap that already finished elsewhere never imports as a ghost card, and a card leaves the dock only once its settlement is genuinely confirmed at depth — not merely because time has passed. Each group carries one **Take over** button. Press it — and confirm that the other machine really is stopped — and this machine adopts **all** of the group's swaps and starts driving them, ledger and all. One kind adopts with a caveat: a **Private (v2)** swap whose payout is pinned to a node wallet this machine doesn't control — or can't positively prove it controls — is taken over **refund-only**, and the activity log tells you so at take-over: this machine won't complete the trade into a wallet it doesn't own, so left alone that swap simply rides to its timelock and refunds to an address this machine *does* own. That verdict isn't final, though — the engine re-checks it on every pass, so attaching the payout wallet to this machine lets the same swap complete normally after all, with no second take-over needed. Nothing is skipped, and nothing is left undriven. All v1 swaps, and v2 swaps

that pay a seed-derived address, adopt and complete without conditions.

> **Note** — If the group holds a swap that hasn't locked anything yet, the confirmation dialog says so honestly: taking over a pre-funding swap is fire-and-forget. It continues only if this wallet can positively verify from the blockchain that its funding is safe to send; if that can't be proven, it ends in a clean abort about 15 minutes after take-over. No funds are at risk either way.

And take-over doesn't set a trap for the other machine, either: if it later comes back online, it checks each of its swaps against the blockchain before acting, sees the ones you finished here, and simply records them as completed — it won't fight you for them or re-broadcast anything.

> **Note** — Upgrading from an older Satchel needs no ceremony here: your **finished** swaps stay in the ledger automatically. A swap that was still **in flight** when you upgraded shows up once under **Another machine** — one **Take over** and it carries on as this machine's swap. (A followed card an older version left stuck showing **active** needs nothing from you either: it sorts itself out on the first pass after the upgrade.)

10.2.2 Dump logs: diagnostics for support

Every swap has a **Dump logs** button — on its card in the active-swaps dock, and on its expandable detail row on the **Swaps** page. Press it and Satchel copies a **diagnostics bundle** for that one swap to your clipboard: the swap's own record plus the engine log lines that mention it. Paste it into a bug report or a message to a developer when something has gone wrong.

> **Tip** — The bundle is **secret-free and safe to share**. Secrets are scrubbed out before it's copied, so it never contains your seed, your recovery phrase, the swap preimage, or any nonces — just the information a developer needs to see what happened.

10.3 Closing the app mid-swap: the exit gate

If you try to close Satchel while a swap is in flight, an **exit gate** dialog stops you and explains the situation: the swap is governed by on-chain timelocks, and the engine must keep running to redeem or refund before the deadline. The gate guards every way out — choosing **Quit** from the tray icon's menu runs through exactly the same dialog, so the tray can never sidestep it.

Your choices:

- **Keep running, close window** (recommended) — the window closes but the engine keeps working in the background until the swap finishes. Reopen Satchel any time to check on it.
- **Cancel** — go back; don't close after all.
- **Force-quit** — stops the engine entirely. This is deliberately hard: you must type the word `QUIT` to confirm, and the dialog warns you in plain terms that doing so can lose funds.

> **Warning** — Choose **Keep running** whenever a swap is live. **Force-quit** kills the engine mid-flight, and with nothing left to redeem or refund before the timelock, funds can be lost. Only force-quit when you have no swaps in flight.

If instead you only have *offers* posted (no live swap), the exit gate is gentler: it lets you withdraw your offers and exit, or keep the engine running so counterparties can still take them while Satchel is closed.

> **Note** — With nothing to protect — no live swap and no posted offers — there's no gate to show, so the window simply closes. If you're only browsing the board without any offers or swaps of your own, closing Satchel is always immediate.

Chapter 11

Your Wallets

The **Wallets** page gives you a quick, at-a-glance balance for each coin you've connected. Click **Wallets** in the left navigation to open it.

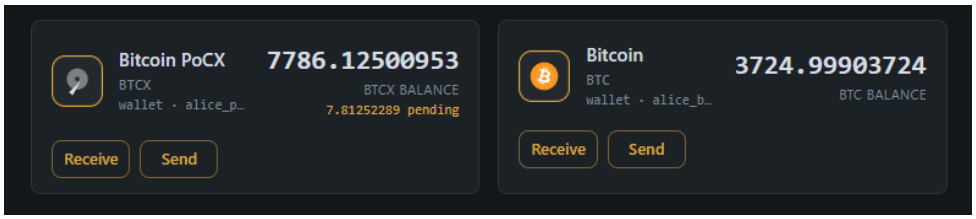

Figure 11.1: The Wallets page: per-coin balances with Send and Receive.

You'll see one card per configured coin, each with the coin's icon, name, symbol, and a large balance figure. The balances come live from your own coin nodes.

The headline figure counts **everything that's yours** — spendable funds plus any money still confirming — so a deposit or a sweep on its way in never makes the card read as an empty wallet. While something is in flight, a small warning-coloured **"*amount* pending"** row appears under the figure and disappears once the coins confirm (freshly mined coins maturing count as pending too). Only the **spendable** part can actually be spent: the send form works from that alone, so a send never trips over money that hasn't confirmed yet.

Each card also tells you **where that coin's wallet lives**:

- A coin connected to **your own node** (RPC) uses the node's wallet. If you've named one (in **Settings** → **Coins**), the card shows **"wallet · {name}"** — every RPC for that coin uses exactly that wallet. If you haven't, a small warning, **"default wallet (not scoped)"**, says the coin falls back to the node's default wallet. On a node with more than one wallet loaded, naming it removes any doubt about which balance you're looking at.
- A coin connected via **Electrum servers** shows **"pact seed wallet"**: there is no node — the wallet lives on your Pact seed (a standard BIP-86 branch of the same recovery phrase), and the servers only supply chain data. They never see your keys.

11.1 Send, Receive and Activity

Every card carries **Receive** (a fresh address each time — old ones keep working, fresh is better for privacy) and **Send**. The send form takes the recipient address and amount, then lets you pick the network fee — added on top of the amount — as **Slow / Normal / Fast** presets priced from the live fee market, or a **Custom** sat/vB rate. When the market has no estimates (a quiet or brand-new chain), the presets grey out and the form falls back to a custom rate at the coin's minimum. A **Review** step shows recipient, amount, estimated fee and total before anything is broadcast — transactions are irreversible, so check the address there. For a node-backed coin these drive the node's own wallet; for an Electrum coin they drive your pact-seed wallet directly.

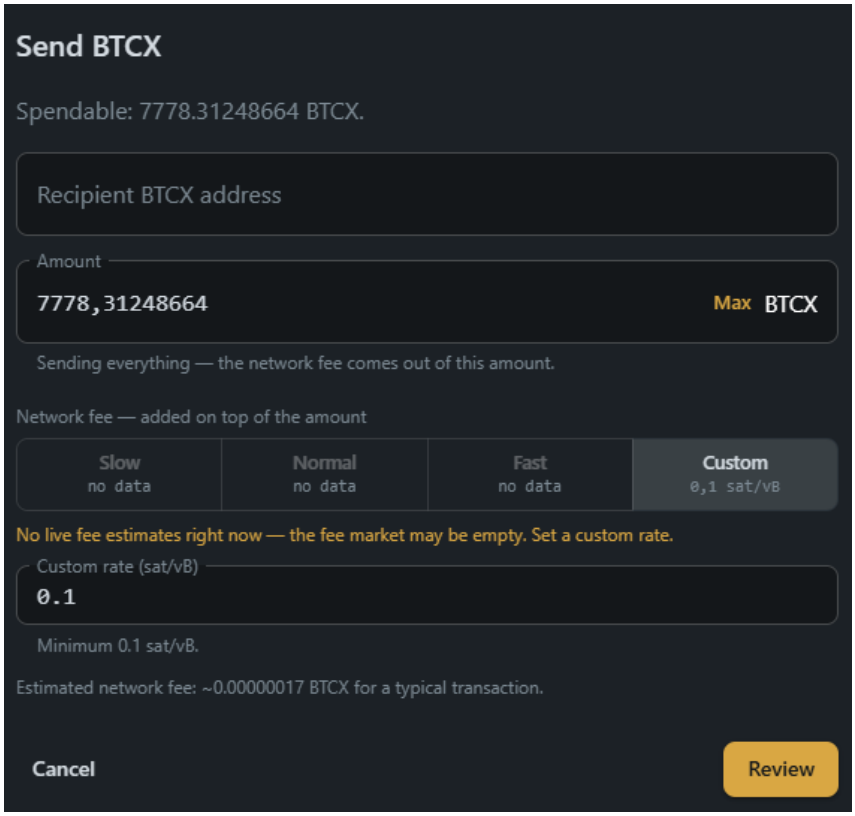

Figure 11.2: The Send dialog: recipient, amount with the Max button, and the network-fee presets.

To empty a wallet, click the **Max** button inside the amount field. It fills in your full spendable balance and flips the send into a *sweep*: instead of being added on top, **the network fee comes out of the amount**, so the wallet ends the send genuinely empty. The confirm step is honest about the arithmetic — the recipient gets roughly your balance minus the fee, and the total equals your balance exactly. Typing in the amount field switches back to a normal send.

> **Tip** — **Max** is the natural way to follow the hot-seed advice below: after a trade, sweep the proceeds to your own cold or main wallet in one go, with no dust left behind from fee guesswork.

Electrum coins add a third button, **Activity** — the wallet's transaction history (direction, amount, fee, confirmations), including anything still pending. A pending send you made carries a **Bump**

action: every send is broadcast replaceable (RBF), so if it's stuck you can re-price it to a higher sat/vB rate and the replacement takes its place. Node-backed coins skip the Activity dialog: your node's own software already keeps that history (and its own fee-bump tooling), and the transactions that matter for trading — funding and settlement — appear on the **Swaps** page with full on-chain detail either way.

> **Note** — One pending row deliberately never offers **Bump**: a transaction that is **funding a live swap**. The engine manages swap fees itself — re-pricing a stuck funding safely on its own (see **Settings** → **Fees**) — and a hand bump there could break the swap's pre-signed settlement. Once the swap finishes, its transactions are ordinary history like any other.

> **Note** — A fine point on **Receive**: a pact-seed wallet hands out fresh addresses until **20** of them are sitting revealed-but-unused, then quietly re-offers the oldest unused one instead of minting more. That cap is what guarantees a wallet restored from your recovery phrase always finds *all* your funds — the restore scan never has a gap too wide to cross. In everyday use you'll never notice it; addresses you've actually received on don't count against it.

11.2 The hot-seed warning

At the top of the page is a warning banner reminding you that a merchant's seed is a **hot** spending seed, not a vault. It holds transit keys for in-flight swaps, which means it has to be available and unencrypted enough for the engine to act quickly.

> **Warning** — Don't park large balances in a merchant wallet. Sweep sizable proceeds to your own cold or main wallet after a trade. The merchant seed is hot by design — treat it as a working float, not savings.

11.3 Loading and error states

The balance on each card updates independently, so one coin having trouble never blocks the others:

- **…** — the balance is still loading.
- **–** — Satchel couldn't read this coin's balance. Hover the dash for a tooltip explaining the error; usually it means that coin's node is unreachable, so start it (or check **Settings** → **Coins**).

If you haven't connected a merchant yet, or haven't set up any coins, the page shows a short prompt with a link to **Coins** in Settings rather than empty cards.

Chapter 12

Contacts

Trade with the same people often enough and you'll want to recognise them. **Contacts** is Satchel's private address book: it lets you put a name to a counterparty so you're not squinting at a string of hex every time. Click **Contacts** in the left navigation to open it.

Every counterparty in Satchel is identified by a cryptographic public key — the same key that draws the little *identicon* and short fingerprint you see on offer cards, on the Swaps page, and in the active-swaps dock. A *contact* maps one of those keys to a **nickname** you choose, an optional freeform **note**, and a **standing**: **Trusted**, **Neutral**, or **Blocked**. That's the whole feature — a way to remember who's who.

> **Note** — Contacts are **local to this device.** They live in Satchel's `satchel.json` settings file and are **never** shared, published, signed, or sent to a relay; the engine never even sees them. Your nicknames are yours alone, and they survive clearing the webview cache. Nobody else learns what you called them, and nobody can read your list off the network.

12.1 What a nickname is — and isn't

A nickname is a convenience label, nothing more. It's shown *alongside* the identicon and fingerprint — it never **replaces** them.

> **Warning** — The identicon and fingerprint remain the real, spoof-proof identity; the nickname is only a note you've pinned to it. Never treat a name as proof of who you're dealing with. If you only went by the label, a hostile party could generate a fresh key, and there is nothing stopping *them* from being the one you happened to nickname "alice." Always check the identicon and fingerprint match the counterparty you expect before you act.

12.2 What standing does — and doesn't

Standing is a *soft, personal* signal — your own memory of how a past trade went. It's deliberately the human judgement the protocol itself leaves out: in Pact, trust is **atomicity only**. The swap either completes for both sides or refunds, no matter who the counterparty is.

So it's worth being honest about the limit here: **Blocked never stops a trade.** Blocking someone only changes your local view and adds warnings — it does not prevent you, or them, from trading. What actually protects you is the atomic swap, not this list. Use standing to keep your own notes tidy, not as a safety mechanism.

12.3 Adding and editing a contact

You don't go to the Contacts page to add someone — you do it wherever you already see them. Click any counterparty's identicon or tag — on a Corkboard offer card, on the **Swaps** page, in the active-swaps dock, or in the take-confirm dialog — and a small menu opens:

- **Add to contacts…** / **Edit contact…** — opens the edit dialog, with a **Nickname** field and a multi-line **Notes** field.
- **Mark as trusted** / **Mark as neutral** / **Block** — sets the standing in one click.
- **Copy public key** — copies the full key to your clipboard.
- **Open in Contacts** — jumps to this contact's row on the Contacts page.

12.4 The Contacts page

The page is a searchable table of everyone you've saved. The search box matches on **nickname, note, or key**, and filter chips — **All / Trusted / Blocked** — narrow the list. The columns are:

Column	What it shows
Identity	The identicon and short fingerprint — the real identity.
Nickname	The label you chose.
Notes	Your freeform note.
Standing	Trusted, Neutral, or Blocked.
Added	When you first saved them.

You can edit a row inline, or remove a contact entirely (Satchel asks you to confirm a removal first). If you haven't saved anyone yet, the page shows a short empty state telling you to click a counterparty's identicon anywhere it appears to add your first one.

12.5 Where standing shows up

Once you've saved a contact, their standing follows them around the app:

- **On the counterparty tag** — wherever a counterparty renders, the tag shows your nickname and a small status dot for their standing.
- **On the Corkboard** — once you've blocked anyone, a **Hide blocked offers** toggle appears above the board. Turn it on to drop blocked makers' offers from the ladder so you don't keep seeing them.
- **At take-confirm** — if you take an offer from someone you've blocked, the confirmation dialog shows a warning ("You blocked this counterparty") and asks you to confirm a second time. It's a personal reminder, not a barrier: blocking does not stop the trade, and the swap is atomic either way.

Chapter 13

Private (Off-Market) Offers

Sometimes you don't want to post to the whole board — you want to trade with one specific person. Maybe you've agreed a price with a friend, or you simply prefer not to advertise. *Private offers* let you do exactly that. Instead of pinning an offer to the Corkboard, you create a small code called a **slip** and hand it to your counterparty directly.

A slip is the same kind of signed offer you'd post publicly — it's just delivered person-to-person instead of broadcast. The swap that results is identical: still atomic, still auto-refunding, still safe.

13.1 Creating a slip

1. Click **Create slip** in the left navigation.
2. Fill in the **same offer form** you'd use to post publicly — coins, amounts, price, swap type, safety timelock. (See the chapter on making an offer for the field-by-field details.)
3. Review and confirm. Satchel produces a slip: a string that begins with `pactoffer1:`, shown in a copyable box.
4. Click **Copy** and send the slip to your counterparty over **any chat** — messaging app, email, whatever you both use. Satchel doesn't send it for you.

The slip box also notes that it **expires in about 24 hours**. After that it's no good and your counterparty will have to ask for a fresh one.

13.2 Taking a slip

If someone sends *you* a slip:

1. Click **Take a slip** in the left navigation.
2. Paste the `pactoffer1:` string into the box.
3. Click **Review & take**. Satchel decodes it, shows you the same take confirmation dialog you'd get from a board offer (amounts, protocol, refund times, fees), and re-checks the maker's signature.
4. Confirm. From there it's an ordinary swap — follow it on the **Swaps** page.

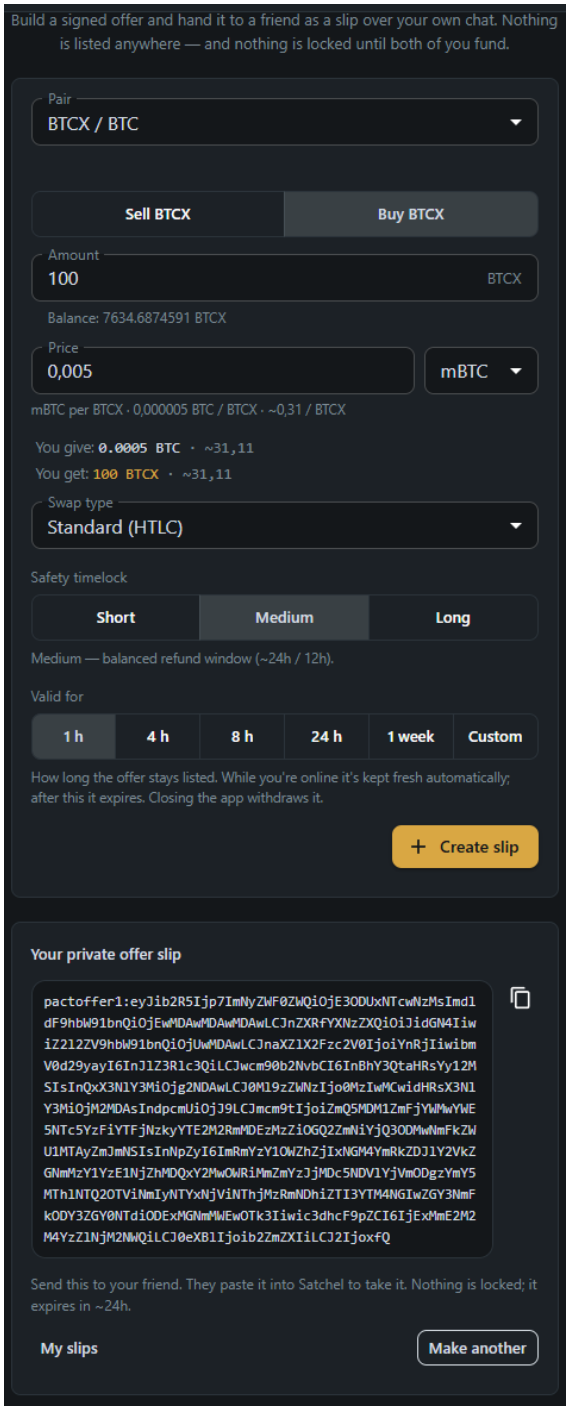

Figure 13.1: Create slip: the offer form produces a copyable pactoffer1: slip.

13.3 Tracking your slips: My slips

Click **My slips** to see the private offers you've handed out. Each card shows the amounts and a countdown to expiry, with a **Cancel** button.

Cancelling stops you honouring that slip: a friend who still holds it can no longer take it. (Slips also expire on their own after roughly 24 hours, so an unused one lapses even if you forget.) If you have no slips out, the screen offers a button to create one.

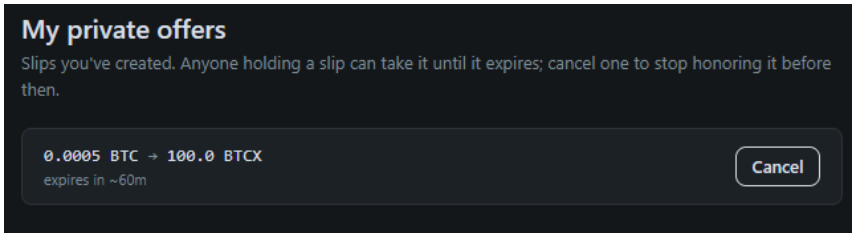

Figure 13.2: My slips: outstanding private offers with expiry countdowns and Cancel.

13.4 A note on slip safety: bearer offers

A slip is a *bearer* instrument: **whoever holds it can take it**, until it expires or you cancel it. There's no name baked into a slip — if you forward it to the wrong person, or it's intercepted, they could take it on the agreed terms.

That sounds scarier than it is, because of what a slip *can't* do:

- **The terms are fixed.** A slip locks in the exact coins, amounts, price, and timelock. Nobody can take it for different terms than you set.
- **The swap is still atomic.** Whoever takes it, the trade either completes for both sides or refunds — nobody can take your funds without giving you theirs.
- **You can cancel.** Use **My slips** → **Cancel** any time before it's taken, and it becomes worthless.

  Tip — Treat a slip like a price you've quoted to one person: only send it to the counterparty you meant, and cancel it if plans change. The worst case is that someone else accepts the *exact trade you offered* — never a worse one.

13.5 When to use private versus public

- **Post an offer (public)** when you want the best chance of a fill and don't mind the offer being visible. The whole board can take it.
- **Create slip (private)** when you've already agreed terms with someone, or you'd rather not advertise — an over-the-counter, trade-with-a-friend deal.

Either way the protection is the same. Private offers change *who can see the offer*, not *how safe the swap is*.

Part III

Transports & Network

Chapter 14

Trading over Nostr & Corkboard

When you post an offer, it has to reach other people somehow. Satchel uses a *noticeboard* for this — a public place where offers are pinned up so others can browse them. Think of a physical cork noticeboard in a town square: you tack up a card saying what you want to trade, people walk past and read it, and someone who likes the deal takes your card down and contacts you.

Satchel can use two kinds of noticeboard, and they work side by side. You don't have to choose — and most people never think about this screen at all, because the default just works.

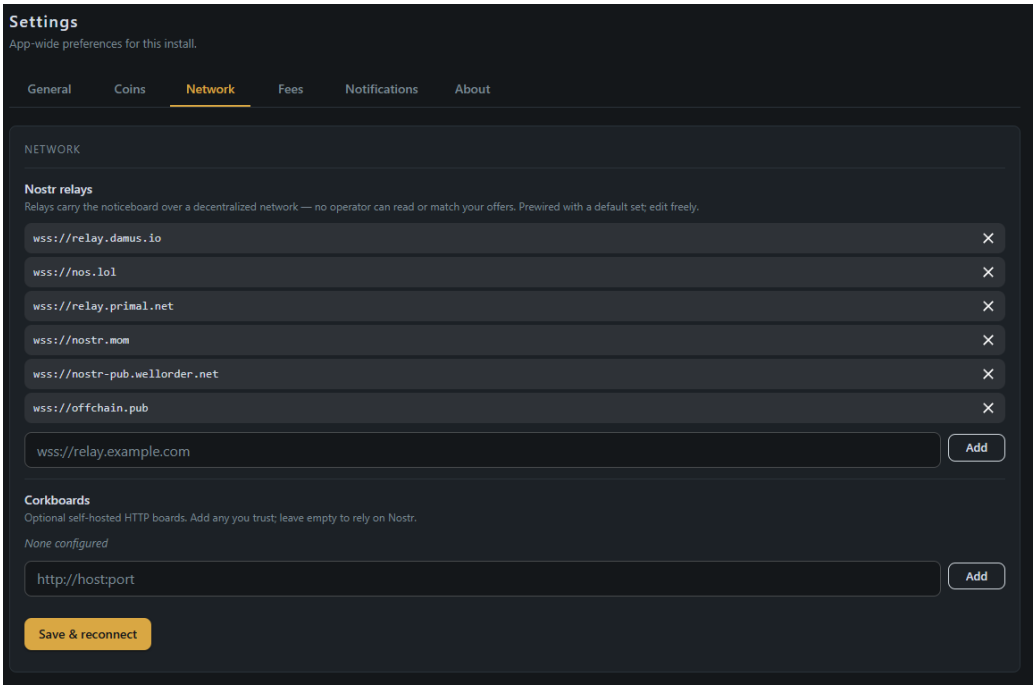

Figure 14.1: Settings, Network tab: the Nostr relays and Corkboards lists.

14.1 The two kinds of noticeboard

14.1.1 Nostr — the default, nothing to run

Nostr is an open, public messaging network made up of independent servers called *relays*. It is not owned by anyone; anybody can run a relay, and they all speak the same simple language. Satchel uses Nostr as its default noticeboard, and **it is ready the moment you install** — there is nothing for you to set up or run.

A fresh copy of Satchel comes with **six recommended relays already wired in**. When you post an offer, Satchel sends a copy to *all* of them at once. This is deliberate. If one relay is slow, offline, or decides to ignore you, the others still carry your offer — so no single server can quietly silence you. People call this *censorship-resistance*, and it is the main reason Nostr is the default.

> **Note** — The six default relays are public, community-run servers on the open Nostr network. You can change the list, add your own, or remove ones you don't like in **Settings** → **Network**. The chapter *"Settings"* shows you how.

You can tell Nostr is working from the small **relay dot** in the header at the top of the window. Green means at least one relay is reachable; amber means none are responding right now. The dot only appears when you have relays configured.

14.1.2 Corkboard — a noticeboard a community can run

A *Corkboard* is a small, self-hostable noticeboard. A community, a club, or a group of friends can run one on a server they control, and then everyone points their Satchel at its web address. It is the same idea as Nostr — a public list of signed offers — but it lives at one address that the community owns and operates.

Corkboard is **opt-in**: a fresh install has none configured. If your community runs one, someone will give you its URL (a web address like `https://board.example.com`). You add it in **Settings** → **Network**, Satchel checks it, and from then on your offers appear there too.

> **Tip** — Corkboard is handy when you want a smaller, more focused marketplace — say, just the members of one community — rather than the wide-open Nostr network. You can run both at once.

14.2 How they work together

The two kinds of noticeboard are equal partners. Here's the simple model:

- **Posting fans out to everything.** When you post an offer, Satchel sends it to *every* noticeboard you have configured — all your Nostr relays *and* every Corkboard at once. You post once; it lands everywhere.
- **Browsing shows one at a time.** When you look at the **Corkboard** screen, you are looking at *one* board's offers. A board selector lets you switch between them — your Nostr offers,

or a particular Corkboard — so you can see who is trading where.

Because posting fans out but browsing is per-board, there is one rule worth remembering:

> **Note** — You and the person you want to trade with only need **one noticeboard in common**. If you both have Nostr enabled (the default), you're already covered. If your community runs a Corkboard and you've both added it, that works too. You don't need to match every board — just one.

14.3 What a noticeboard can and cannot see

This is the part that matters most, so it's worth being clear. A noticeboard — whether a Nostr relay or a Corkboard — is a **dumb pinboard**. It carries two kinds of thing, and nothing else:

1. **Your public offers**, which are signed by you. An offer says "I'll give X for Y, valid for an hour" — and it is *meant* to be public, so people can find and take it. Anyone can read it; only you could have created it.
2. **Sealed messages** passed between two traders mid-swap. These are *encrypted* before they ever leave your machine. The board stores them and hands them on, but it cannot open them.

What a noticeboard **never** sees:

- **Your keys.** Your private keys and recovery phrase never leave your computer. Nothing on a board can derive them.
- **Your coins.** A noticeboard holds no money. It cannot touch your funds, and there is nothing in your wallet for it to take.
- **The contents of sealed messages.** Mid-swap messages are encrypted to the recipient only. The board sees scrambled bytes — not who said what.

And just as importantly, a noticeboard **cannot match you with anyone or execute a trade**. It has no power to pair offers, move coins, or settle a swap. All of that happens directly between you and your counterparty, enforced by the blockchains themselves (the chapter *"Understanding Atomic Swaps"* explains how). The board is only a place to find each other.

> **Warning** — Treat every offer on a noticeboard as a *claim*, not a promise. The board does not vet anyone. Your safety never comes from trusting the board or the other trader — it comes from the swap itself, which is built so that either both sides get paid or neither does. Always confirm the amounts and the safety timelock in the take dialog before you commit.

14.4 Do I need to think about any of this?

Usually not. For most people the honest answer is: **install Satchel and start trading.** Nostr is on by default with six relays prewired, so you have a working noticeboard from minute one. You only need to open **Settings** → **Network** if you want to add a community Corkboard, trim the relay list, or turn Nostr off entirely — all of which the next chapter, *"Settings"*, walks through.

Chapter 15

Settings

Everything you can configure in Satchel lives behind the **Settings** gear, which you'll find in the top-right header and again at the foot of the left-hand navigation. Settings is organised into six tabs — **General**, **Coins**, **Network**, **Fees**, **Notifications**, and **About** — and this chapter takes them in turn.

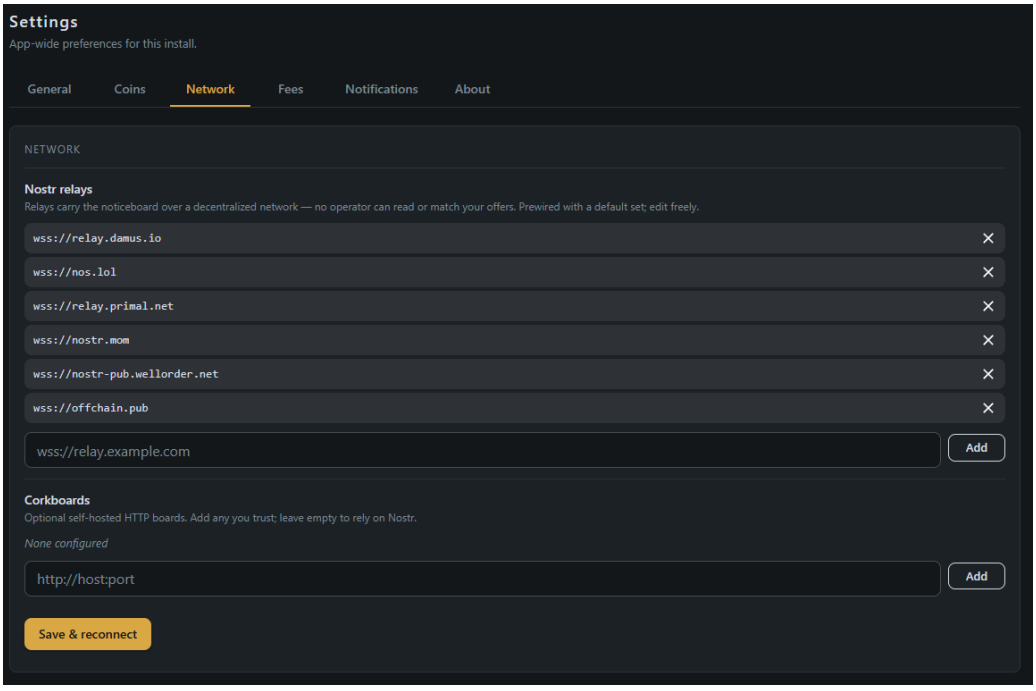

Figure 15.1: The Settings screen with its tabs.

Most of what's here you'll set once and forget. None of it touches your coins or your recovery phrase — these are preferences and connection details, nothing more.

15.1 General

The **General** tab holds your look-and-feel preferences.

- **Theme** — choose **Light**, **Dark**, or **System**. **System** follows whatever your computer is set

to (dark in the evening, light by day, if your OS does that). The change applies instantly and is remembered next time you open the app.

- **Language** — selects the app's display language. Satchel ships in **26 languages**, each listed under its own native name; pick one and the app switches straight away. (There's also a globe picker in the header for changing language on the fly.)

  Tip — There's also a quick theme and language reach from the header, but the **General** tab is the canonical place to set them.

  Note — There's no "browse mode" to turn on: Satchel always lets you browse the **Corkboard** freely, whether or not you've connected any coins. Trading is gated per action instead — posting, taking, and funding need **two live coins**, and each nudges you to **Settings** → **Coins** when you reach for it.

15.2 Coins

The **Coins** tab is where you tell Satchel how to reach the cryptocurrency nodes you trade with — your Bitcoin node, your BTCX node, and so on. Because this is a big topic with its own setup flow, it has a dedicated chapter.

> **Note** — See the chapter *"Setting Up Your Coins"* for the full walkthrough: adding a coin, entering its connection details, validating it, and reading the status pills and trading-pair list. The same screen appears both there and here under **Settings** → **Coins**.

In short: each coin shows a status pill — **Not set up**, **Connected**, or **Connection error** — and a **Set up** or **Edit connection** button. Below the coins, a **Trading pairs** list tells you which pairs are ready to trade.

15.3 Network

The **Network** tab controls your noticeboards — the places your offers are posted and browsed. (For the bigger picture of how these work, see the chapter *"Trading over Nostr & Corkboard"*.) It holds just two editable lists. (The old read-only "network mode" row is gone — which network you're on is fixed when Satchel launches and is shown in the top-bar badge instead.)

15.3.1 Nostr relays

This is the list of *Nostr relays* — the public servers that carry your offers on the open Nostr network. A fresh install arrives with six recommended relays already in the list, so you normally don't need to touch this.

- To **add** a relay, type its address. A valid relay address starts with `wss://` (for example `wss://relay.damus.io`). Satchel checks the format as you type.
- To **remove** one, delete it from the list.

> **Note** — If you clear the Nostr relays list completely, **Nostr is turned off**. That's a valid choice — for instance, if you only ever trade over a private Corkboard — but with an empty list your offers won't reach the open Nostr network at all.
>
> **Tip** — This is the place you **add and remove** relays, but to *watch* how they're doing — which are connected, their latency and uptime — open the top-level **Network** screen in the left navigation (its Nostr tab; described in the *Tour of the Interface* chapter). That screen is monitor-only.

15.3.2 Corkboards

This is the list of *Corkboard* noticeboards — the self-hostable boards a community might run. A fresh install has none; you add one only if you have its address.

- To **add** a Corkboard, type its web address. A valid Corkboard address starts with `https://` (or `http://`). If the list is empty, the field reads **None configured** — which is perfectly fine.
- To **remove** one, delete it from the list.

15.3.3 Saving your changes

Both lists are edited together. When you're happy, press **Save & reconnect**. Satchel saves the new relay and Corkboard lists and reconnects to them right away — you'll see the header's relay dot update to reflect what's now reachable.

> **Tip** — If a freshly added relay or board shows amber or red after saving, double-check the address for a typo. The chapter *"Troubleshooting"* has a short section on noticeboards that won't connect.

15.4 Fees

The **Fees** tab holds one advanced, optional section — **Fee bumping** — that sets the limits Satchel works within when it raises an on-chain fee to get a stuck swap transaction confirmed. Satchel now does the bumping automatically, tracking the going fee market for you rather than stepping up by a fixed amount, so these are just the guard-rails. **You don't need to touch this to trade.** The defaults are sensible; they're safety-versus-cost trade-offs for the rare swap that needs a nudge.

A couple of things to know before you change anything:

- The settings apply to your **active merchant**.
- New values affect **future** bumps only. A swap that's already funded keeps the policy it was funded under, so changing these won't disturb a trade in flight.

The two knobs are:

- **Max feerate (sat/vB)** — the ceiling on **funding** fee bumps, so a runaway fee market can never drain you. Redeem and refund bumps deliberately ignore it — near a timelock they

pay whatever it takes (capped only by the value at stake), so a leg is never lost to a fee cap. Range 1–500; **default 500** (also the hard system maximum).

- **Funding bump reservation (×)** — applies to funding bumps only, and serves two roles: how much extra balance the funds check sets aside as headroom for a possible future bump, *and* the cap on that bump (this multiple of the funding's original feerate). Range 1–100; **default 3**.

Press **Save** to apply your changes — they take effect **live, with no restart** — or **Reset to defaults** to put both back the way they came.

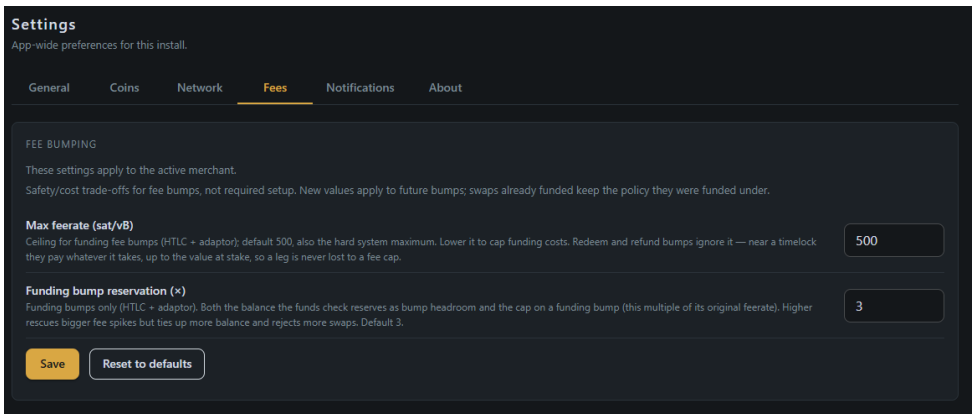

Figure 15.2: The Fees tab: the fee-bumping knobs with Save and Reset to defaults.

> **Note** — If you've never heard of fee bumping or RBF, that's fine — leave this tab alone. Satchel handles fees for you out of the box; these knobs exist only for people who want fine control over the cost of a stalled swap.

A word on how the automatic side behaves, since it shows up in the feerates on your swap cards. A **claim** — the transaction that collects the coins you're owed — is in a genuine race against the swap's deadline, so it's priced to confirm at the same brisk pace as the swap's funding right from the start, and the engine escalates it as the deadline nears: in roughly the last hour and a half it re-checks the fee market on every pass and bumps whenever the market has moved above what the transaction pays. A **refund** never gets that treatment, on purpose — after the timelock, nobody else can spend those coins, so there's no race and a refund can afford to wait for a cheap block.

15.5 Notifications

Swaps take a while and run on their own — so the **Notifications** tab lets your operating system tap you on the shoulder when one hits a milestone while Satchel is in the background. **Nothing fires while you're looking at the window**: the in-app activity log and swap cards already narrate everything, so notifications only speak up when you're elsewhere.

At the top sits **Enable notifications** — the master switch that turns every notification below on or off. Under it, five per-event toggles, all **on** by default:

- **Swap started** — someone took your offer, or a maker accepted your take.

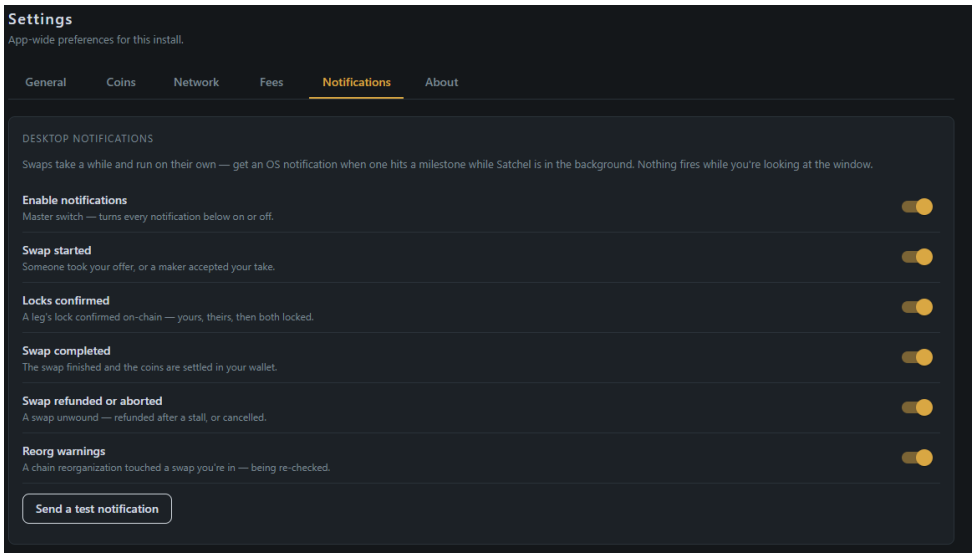

Figure 15.3: The Notifications tab: the master switch, the five event toggles, and the test button.

- **Locks confirmed** — a leg's lock confirmed on-chain: yours, theirs, then both locked.
- **Swap completed** — the swap finished and the coins are settled in your wallet. (This fires only at the true end — the finalizing phase stays silent.)
- **Swap refunded or aborted** — a swap unwound: refunded after a stall, or cancelled.
- **Reorg warnings** — a chain reorganization touched a swap you're in and its confirmations are being re-checked.

The notification text is the same plain-language narration you see in the activity log, so the story reads identically wherever you catch it.

A **Send a test notification** button lets you check the plumbing. If nothing appears, the most common cause is the operating system itself blocking notifications — allow Satchel in your system notification settings (and check Do Not Disturb).

> **Tip** — Notifications pair naturally with the **tray icon** and the exit gate's **Keep running** option: close the window mid-swap, let the engine work in the background, and the milestones come to you. The tray icon (see the *Tour of the Interface* chapter) shows how many swaps are in flight, and its **Quit** goes through the same fund-safety gate as closing the window.

15.6 About

The **About** tab is your reference corner.

- **Version** — the exact Satchel version you're running, with an **Up to date** badge or an update notice. If a newer release exists, Satchel tells you here (and with a small badge by the logo in the navigation); use **Check for updates** to look again.
- **This machine** — this install's short id, shown as **This machine · M-xxxx**. It matters only if you run the same recovery phrase on more than one machine: each machine drives its

own swaps, and this label is how the others refer to this one in the active-swaps dock. See *"One seed on more than one machine"* in the *Backup, Seeds & Safety* chapter.

- **Where your keys live** — a short trust note reminding you that your keys and recovery phrase stay on your own machine and are never sent to Satchel, to any noticeboard, or to anyone else.
- **Risk disclaimer** — the self-custody notice. Satchel is provided without warranty; you alone hold your keys and are responsible for safeguarding your recovery phrase and the funds you trade.

15.7 Where your settings are stored

All of these preferences — your theme, language, coin connections, relay list, Corkboard list, and notification choices — are saved in a small file named `satchel.json` on your own computer. Nothing is stored in the cloud or on any server.

> **Note** — `satchel.json` holds *settings only* — never your recovery phrase or any private key. Those are handled separately and far more carefully; the chapter *"Backup, Seeds & Safety"* explains exactly where they live.

Part IV

Safety & Reference

Chapter 16

Backup, Seeds & Safety

This is the most important chapter in the book. Trustless trading puts you in complete control of your money — and that control comes with responsibility. No company holds your coins, so no company can lose them, freeze them, or hand them to a thief. The flip side is that **no one can recover them for you, either.** Almost all of your safety comes down to one small list of words and a few simple habits. Please read this chapter slowly.

16.1 Your recovery phrase

When you create a trading identity, Satchel shows you a *recovery phrase* — a list of 12 or 24 ordinary words, in a specific order. (You may also hear it called a *seed* or *seed phrase*; they're the same thing.) Those words *are* your wallet. Everything Satchel does on your behalf — your trading keys, your addresses, your ability to refund a swap — is mathematically derived from them. Lose the words and you lose access; let someone else have them and they have your money.

There are exactly two rules. They are non-negotiable.

> **Warning — Write your recovery phrase down on paper the moment Satchel shows it to you**, and store it somewhere safe and private — ideally more than one place. This is the *only* way to recover your funds if your computer is lost, stolen, or breaks. Do it before you click past the screen.
>
> **Warning — Never type or paste your recovery phrase into any website, chat, email, or "support" form. Ever.** Anyone who gets those words can take everything, instantly and irreversibly. No genuine support person, and no one from this project, will *ever* ask you for it. Satchel itself only ever asks for it on its own first-run screen, never on a web page.

A few practical do's and don'ts:

- **Do** write it by hand on paper (or stamp it into metal, if you want it fireproof). Pen and paper beats a screenshot.

- **Don't** take a photo of it, save it in a notes app, email it to yourself, or store it in a password manager that syncs to the cloud. Each of those is a copy a thief could reach.
- **Do** double-check you copied every word correctly and in order before you continue. Satchel asks you to confirm a few of the words for exactly this reason.

16.2 Encrypting with a passphrase

When you set up your identity, Satchel offers to protect your seed with a *passphrase* — a password of your choosing. This is **optional but strongly recommended**, especially on a computer others might use.

Here's what the passphrase does and doesn't do:

- **It protects the copy of your seed on this computer.** With encryption on, your seed is stored scrambled, and Satchel asks for your passphrase once each session to unlock it. Someone who steals your laptop can't trade or drain your funds without it.
- **It is not your recovery phrase, and it is not a backup.** The passphrase only guards the local copy. Your recovery phrase is still what restores your funds anywhere.
- **Even without a passphrase, your seed isn't left as plain text.** On Windows and macOS, Satchel locks the on-disk copy with a key held in your operating system's secure store, so the file is useless if copied off your machine. That key stays on this computer, so it doesn't stop someone already logged in as you — a passphrase is what adds that extra lock. (On Linux the no-passphrase seed is only lightly scrambled; treat it as unencrypted and use a passphrase if the machine isn't solely yours.)

  Warning — If you forget your passphrase, **Satchel cannot reset or recover it** — there's no "forgot password" link, by design. You would need to restore from your recovery phrase to set a new one. So: choose a passphrase you'll remember, and keep your written recovery phrase safe as your ultimate fallback.

  Note — Because that no-passphrase lock lives in *this computer's* secure store, there's one situation where Satchel can find your stored seed but **no longer unlock it**: the data folder was moved to a new machine, or the system keychain was reset. Satchel doesn't fail cryptically — at startup it opens a **guided re-import dialog** that explains exactly this and asks for your recovery phrase. Re-entering the phrase sets the seed up again on this machine, and **your funds and swaps are not affected**. (This is your written-down phrase doing precisely the job it exists for.) If any swaps were in flight before the move, they reappear under **Another machine** in the swaps dock — one **Take over** resumes them; see *"One seed on more than one machine"* below.

16.3 What the engine holds, and what never leaves your machine

Behind Satchel runs *the engine* (its technical name is `pactd`). The engine is the trustworthy worker that does the real swap work on your computer:

- It holds your **keys**, unlocked in memory for the session, and signs transactions with them.
- It **builds and watches** your swaps from start to finish, broadcasting the right transaction at the right moment.
- It **auto-refunds** you if a swap doesn't complete (more on that below).

Crucially, all of this happens **locally, on your machine**. What is *never* sent to any server, noticeboard, or other person:

- your recovery phrase and private keys;
- your passphrase;
- the contents of your wallet.

The only things that ever go out are your **public, signed offers** (which are meant to be seen) and **encrypted messages** to your counterparty mid-swap (which the noticeboard can't read). See the chapter *"Trading over Nostr & Corkboard"* for exactly what a board can and can't see.

16.4 Keep Satchel running until a swap completes

This rule prevents the most avoidable kind of loss.

> **Warning** — **While a swap is in progress, keep Satchel (and your nodes) running.** A swap is a sequence of timed on-chain steps, and the engine has to be awake to broadcast each transaction — including the refund — at the right moment. If you close everything mid-swap, those steps can't fire on time.

A swap isn't instant: each side locks funds on a blockchain, waits for confirmations, and then claims. The engine babysits all of this for you, but it can only do so while it's running. This is why, when you try to quit during a live swap, Satchel stops you with a clear warning and offers to **Keep running** in the background instead. Take that option. (Quitting from the tray icon's menu goes through the same warning — no exit path skips it.) While the engine works in the background, the tray icon keeps watch and swap milestones arrive as desktop notifications, so backgrounding a swap never means losing sight of it.

> **Tip** — If you genuinely must shut the machine down, the **Keep running** choice on quit lets the engine continue in the background. And even in the worst case, your seed and the swap's timelocks protect your funds — see *"If your machine dies mid-swap"* below.

16.5 A note on hot seeds

When you open the **Wallets** screen, a banner reminds you that these balances belong to your own nodes' wallets, and nudges you about *hot seeds*. A "hot" seed is simply one that lives on an internet-connected, running machine — which is exactly what a live trader needs. That's normal and fine, but it carries the usual caution:

> **Warning** — Don't keep more on a hot, always-on trading machine than you're comfortable having exposed. Treat your trading balance like the cash in your pocket, not

your life savings. Keep long-term holdings in cold storage (an offline wallet), and move funds to your trading machine as you need them.

16.6 One seed on more than one machine

You can restore the same recovery phrase on a **second machine** — a warm standby, a backup box, a replacement laptop — and that is now **safe**. Each install gets its own short identity: open **Settings** → **About** and you'll see **This machine · M-xxxx**, this computer's label. Each machine drives only its own swaps:

- **Another machine's swaps show up read-only.** In the active-swaps dock they appear grouped under **Another machine · M-xxxx**. This machine watches them on-chain but never acts on them, and they stay out of its Swaps ledger.
- **Take over only when the other machine is really stopped.** Each group carries one **Take over** button, behind a confirmation that spells out the condition: *only if that machine is stopped*. Confirm, and this machine adopts those swaps and drives them to completion — the standby becoming the main, in one click. (One special case: a **Private (v2) swap** whose payout is pinned to a node wallet this machine doesn't control — or can't prove it controls — is still adopted, but **refund-only**, and the activity log says so at take-over: this machine won't complete the trade into a wallet it doesn't own, so left alone that swap rides to its timelock and refunds to an address this machine *does* own. The engine re-checks on every pass, so attaching the payout wallet later lets the swap complete normally after all. Nothing is skipped or left undriven. All v1 swaps, and v2 swaps that pay a seed-derived address, adopt and complete without conditions.) Make absolutely sure the dead machine is genuinely off first: two machines driving the same swap at once can lose money, and the confirmation is your promise that it can't happen.

  Note — The confirmation dialog is also honest about swaps that haven't locked anything yet: taking one of those over is fire-and-forget. It continues only if this wallet can positively verify from the blockchain that its funding is safe to send; if that can't be proven, it ends in a clean abort about 15 minutes after take-over. No funds are at risk either way.

  Note — The road back is safe too. If the original machine comes back after its standby finished a swap, it doesn't blindly resume from its old records: on startup it checks each of its swaps against the blockchain first, and a swap the standby already settled simply shows its true final state — **Completed** or **Refunded** — in the ledger. Nothing is re-broadcast, and nothing lingers as a stuck active card or a doomed refund attempt.

  Warning — Machines sharing a seed share **one balance**. This is a failover arrangement, not doubled liquidity: running two machines does not give you two pots of funds to trade from, and trading actively from both at once means they compete for the same coins. Use the second machine as a standby and a safety net, not a second trading desk.

16.7 If your machine dies mid-swap

Hardware fails, power cuts happen, laptops get left on trains. Three things protect you, and it's worth knowing what each one covers:

1. **Your recovery phrase protects your identity and keys.** Every key is derived from your seed, so restoring from your recovery phrase on a new machine brings your trading identity back and lets you trade again.
2. **A quiet backup of your in-flight swaps rides along on the relays.** While a swap is under way, Satchel's engine also backs up just enough of its state to the Nostr relays you're already connected to — **encrypted so only you can read it** — so a swap survives even if the machine that ran it never comes back. Nobody but you can decrypt these backups; not the relay, not anyone else. This is what lets a *brand-new* machine, holding only your recovery phrase, pick a swap back up.
3. **The timelock guarantees a refund — as long as some machine of yours is watching.** Every swap has a built-in deadline called a *timelock*. Funds you locked become refundable to you after that deadline, and **nobody can take them before then.** The refund still has to be broadcast by a running engine that knows about the swap, which is exactly what the relay backup restores.

Putting those together, if a machine dies mid-swap:

- **Install Satchel on a new (or the same, repaired) machine and restore your recovery phrase.** Your identity and keys come back immediately.
- **The engine notices if it finds rescuable swaps.** On unlocking your restored identity, the engine checks the relays for any in-flight swap it doesn't already know about, and — if it finds one — logs a warning rather than silently resuming it. This is deliberate: if the old machine is still alive and quietly working the same swap, having two machines drive it at once could cost you money. **Only proceed once you're sure the old machine is genuinely retired** (wiped, destroyed, or definitely never coming back online).
- **Completing the restore is currently a technical, one-time step**, run through the engine's command-line tool (`pact-cli restore`) rather than a Satchel button — this handbook's companion, the *Pact Developer Handbook*, covers it, or the chapter "Where to Get Help" points you at the community channels if you'd rather have someone walk you through it. Once restored, the engine drives the swap on its own exactly like any other — completing it if it still can, or refunding you once the timelock allows — with no data folder to carry over by hand.

  Warning — This safety net only covers swaps **started after this feature shipped**. A swap that was already in flight when you upgraded was never backed up to the relays and is not retroactively rescuable — for those, the data folder is still what carries the swap across machines (see below). And in the rare case where you were the **maker of a Private (Taproot)** swap and your machine died in the narrow window after accepting but before the final signatures were assembled, the swap cannot be completed even after restoring — the assembled signatures are the one piece of

data that can't be recreated from your seed. Your funds are never at risk either way: the timelock refund is always the fallback.

Note — Keeping Satchel's **data folder** as an actual backup (copying it alongside your recovery phrase) still works too, and restores a swap immediately with no relay round-trip or waiting period — useful if you're migrating a working machine deliberately rather than recovering from a loss. But for an unplanned loss, you no longer *need* it: the relay-based rescue above is the automatic safety net, keyed off your recovery phrase alone.

Note — Three things, three jobs: the **timelock** protects funds *locked in a swap*; your **recovery phrase** protects your *identity and keys* and, via the relay rescue, lets a new machine rediscover an *in-flight swap* too; your **data folder**, if you happen to still have it, restores a swap instantly with no waiting. Back up the phrase always — it's now doing more work than ever. Without it, a lost computer is a disaster — write it down.

Chapter 17

Understanding Atomic Swaps

You can trade happily with Satchel without ever reading this chapter — the app handles the machinery for you. But many people sleep better once they understand *why* a trade with a stranger is safe even though there's no exchange in the middle. That's what this chapter is for. It's conceptual and friendly; if you want the real cryptography, the companion *Pact Developer & Integrator Handbook* has every byte.

17.1 The problem: trading without trust

Imagine you want to swap some BTCX for someone else's BTC. You've never met them. Whoever sends first is taking a risk: what if the other person just keeps the coins and vanishes? On a traditional exchange, a company sits in the middle holds both sides' money and makes sure the trade is fair — but then you have to trust *that company* not to lose, freeze, or steal your funds.

An *atomic swap* removes the middleman without adding that risk. "Atomic" means **all-or-nothing**: the trade is built so that either *both* sides receive their coins, or *neither* does. There is no in-between where one person walks away with both halves.

17.2 How "all-or-nothing" works

The trick is to **link the two payments together** so that claiming one automatically makes the other claimable. Picture two locked boxes — one holding your BTCX, one holding their BTC — fitted with a clever shared lock. The moment one box is opened, opening it reveals the secret that unlocks the other. Neither box can be opened on its own.

So the sequence is safe at every step:

- Both sides lock their coins into these linked arrangements on their respective blockchains. Locked funds can't be spent by anyone yet.
- When the first party claims their side, the act of claiming **reveals a secret** on the blockchain.
- The other party sees that secret and uses it to claim their side.

If everyone plays along, both get paid. And if someone tries to cheat by claiming their half without letting the other claim theirs — the maths simply doesn't allow it. The very action that releases one half is what makes the other half releasable.

> **Note** — Crucially, the blockchains themselves enforce all of this. No noticeboard, no server, and no person can override it. That's why your safety never depends on trusting the board or the other trader.

17.3 The safety net: timelocks

There's one more thing to handle: what if the other side simply does nothing — locks up, gets cold feet, or disappears halfway through? You don't want your coins stuck in a locked box forever.

That's what the *timelock* is for. Every locked box has a **built-in deadline**. If the swap hasn't completed by then, the lock releases your funds **back to you**. You get a full refund, automatically, with no cooperation needed from the other side.

> **Tip** — This is why Satchel asks you to pick a **safety timelock** (Short, Medium, or Long) when you post or take an offer. It's choosing how long to wait before that refund deadline kicks in. **Medium** is a sensible default. Longer gives a slow swap more breathing room; shorter gets your money back sooner if something stalls.

The two halves of a swap have slightly different deadlines on purpose — the engine arranges them so that the person who could be left exposed always has time to react or refund. You don't have to think about this; Satchel sets it up correctly and watches the clock for you. (The one thing it needs from you is to **stay running** until the swap finishes — see the chapter *"Backup, Seeds & Safety"*.)

So the worst realistic case isn't "I lose my coins." It's "the trade didn't happen, and after the timelock I got my coins back." That's the confidence atomic swaps are designed to give you.

17.4 Two flavours of swap

Satchel knows two ways to build a swap. The cryptography differs under the hood, but what matters to *you* comes down to privacy and how the trade looks on the blockchain. You usually don't pick — Satchel chooses the best method a given trading pair supports — but you'll see them labelled on offers.

17.4.1 Standard (HTLC)

The classic, well-proven method, labelled **Standard (HTLC)** on offers. It uses a special kind of locked output that anyone inspecting the blockchain can recognise as a swap. It's robust, battle-tested, and the default for the first pair, BTCX ↔ BTC.

17.4.2 Private (Taproot)

A newer method, labelled **Private (Taproot)**. When everything goes smoothly, a swap built this way looks like an ordinary, everyday transaction on the blockchain — there's no obvious "this was a swap" fingerprint for outside observers to see. It uses a more modern Bitcoin feature called Taproot to achieve this.

> **Note** — Both methods are equally *atomic* — equally all-or-nothing, equally protected by timelocks. The difference is privacy and on-chain footprint, not safety. Where a pair supports both, the offer form lets you choose; where it supports only one, Satchel picks it for you.

17.5 Want the real cryptography?

This chapter is the comfortable, plain-language version. If you want to know exactly how the secret is constructed, how the locking scripts are written, how the Taproot signatures combine, and how the timelocks are computed down to the second, that all lives in the **Pact Developer & Integrator Handbook**. It's written for developers, but it's there if curiosity strikes.

> **Note** — Satchel is live: both swap types — Standard (HTLC) and Private (Taproot) — are reviewed and running on mainnet. The all-or-nothing guarantee is enforced by the chain itself, but you alone hold your keys — so lean on the safety habits in *"Backup, Seeds & Safety"*.

Chapter 18

Troubleshooting

Things occasionally don't go to plan — a node is down, an address has a typo, a swap seems stuck. This chapter walks through the situations you're most likely to meet and how to put them right. Work top to bottom; each entry is independent, so jump to the one that matches what you're seeing.

> **Tip** — The header at the top of the window is your dashboard. The engine-reachability dot, the Nostr relay dot, the per-coin health glyphs, and the live swap count all tell you at a glance what's healthy and what isn't. Hover any of them for a tooltip.

18.1 "Engine not running" or disconnected

If Satchel shows a disconnected state — the engine indicator is red, and screens can't load data — the app can't reach its swap engine. The usual causes:

1. **A node is down.** The engine talks to your cryptocurrency nodes (Bitcoin, BTCX, and so on). If one isn't running, start it and give it a moment to come up.
2. **The wrong RPC credentials.** If a node is running but the engine can't log in to it, the connection details are off. Open **Settings** → **Coins**, edit the coin, and re-check the host, port, and authentication (cookie file path, or username/password). See the chapter *"Setting Up Your Coins"*.
3. **The engine was stopped or crashed.** If the engine Satchel launched died and came back, Satchel **reconnects to it automatically** the next time a screen asks for data — you don't need to restart the app, just give it a moment (or click into any page) and the dot turns green again. If it stays red, closing and reopening Satchel relaunches the engine cleanly.

   Note — Satchel needs the engine to be reachable before it can show balances, offers, or swaps. Fixing the engine connection usually fixes several symptoms at once.

18.2 Satchel won't start — "another network's engine is on this port"

If you run **more than one Satchel** — say one on the practice network and one on mainnet — they each start their own engine, and each engine needs its **own port** to listen on. If a second Satchel finds that another network's engine is **already using the port** it was told to use, it now **refuses to start** and shows a clear error rather than quietly latching onto the wrong engine (which could have it trading on a network you didn't mean).

This is a deliberate safety stop. To clear it, give each Satchel its own port — or shut the other one down:

1. **Use a distinct port per network.** In the `satchel.json` settings file for the one that won't start, set its **`listen`** port to the value for its network: **9739** for regtest, **9738** for testnet, **9737** for mainnet. With each network on its own port, they stop colliding.
2. **Or stop the other instance** that's already holding the port, if you didn't mean to have two running.
3. Relaunch Satchel.

   Note — This only comes up when two Satchels (or their engines) try to share one port. A single Satchel never hits it. The chapter *"Settings"* explains where `satchel.json` lives.

18.3 A coin shows a connection error

In **Settings** → **Coins**, each coin carries a status pill. **Connection error** (rather than **Connected**) means Satchel reached the engine but the engine can't talk to that particular node.

To fix it:

1. Confirm the node itself is running and has finished starting up.
2. Open the coin's **Edit connection**, check the host, port, and authentication, then press **Validate node**.
3. A green "Genesis matched" verdict means it's good — press **Save**. A rejection means the details point at the wrong node or network; correct them and validate again.

The chapter *"Setting Up Your Coins"* covers the validation flow and what each field means in detail.

18.4 "Fewer than 2 coins connected"

Satchel needs at least **two live coins** before you can trade — after all, a swap is an exchange between two of them. There's no wall keeping you out: you land in the app whatever your coin count, and can browse freely. What's gated is the money-moving actions, **per action** — **Post an offer** and **Create slip** show a soft "Set up two coins to trade" prompt, and a board offer's

Take button stays disabled until both of its coins are live. If an action is out of reach, one or both of your coins isn't fully connected.

- The per-coin health glyphs in the header tell you how many are live. Pick a coin that isn't connected and complete its setup.
- A coin counts as *live* only when it's both configured **and** showing **Connected**. A coin with a **Connection error** doesn't count — fix it as above.

Once two coins are green, those actions unlock.

18.5 No offers on the Corkboard

If the **Corkboard** screen is empty, it's usually one of these:

- **No board or relay configured.** If you see "No Corkboard connected," open **Settings** → **Network** and make sure you have at least one Nostr relay or one Corkboard. A fresh install ships with six Nostr relays, so this is rare unless you cleared the list.
- **You're browsing the wrong board.** The board selector at the top switches between your noticeboards. If your counterparty posted on a specific Corkboard, select that board.
- **The wrong pair, or coins you haven't connected.** The Corkboard filters to the trading pair you've selected, and only shows pairs whose coins you've set up. Try another pair, or connect more coins in **Settings** → **Coins** to see their offers.
- **It's just quiet.** Sometimes nobody's posted recently. Consider posting your own offer — see the chapter on posting offers.

18.6 A relay shows amber or red

The Nostr relay dot in the header turns **amber** when none of your relays are reachable, and the **Settings** → **Network** list can flag individual ones. Usually:

- **A typo in the address.** Relay addresses must start with `wss://`. Re-check the one you added.
- **That relay is temporarily down.** Public relays come and go. If you have several configured (the default has six), the others keep you running — one amber relay isn't a problem. If *all* are amber, check your internet connection.
- Press **Save & reconnect** after any change to retry the connections.

18.7 A swap looks stuck

A swap that seems frozen is usually just *waiting* — for blockchain confirmations, or for the other side to take their step. Swaps are not instant.

1. **Keep Satchel and your nodes running.** This is the single most important thing. The engine needs to be awake to broadcast the next step — including the refund — at the right time.

2. **Let it run its course.** Check the **Swaps** screen; each row shows a plain- language narration of where it is. If the other side has gone quiet, the engine is already watching the *timelock* on your behalf.
3. **Trust the refund.** If the swap can't complete, the timelock guarantees your locked funds come back to you automatically once its deadline passes. You don't need to do anything except keep the app running. See the chapter *"Backup, Seeds & Safety"*.

   Warning — Do **not** close Satchel to "reset" a stuck swap. The engine has to be running to finish or refund it. If you must shut down, choose **Keep running** on the quit dialog so the engine continues in the background.

18.8 A swap is stuck at funding / my wallet was locked

If a swap's progress line reads **"Locking your {coin} — unlock wallet if stalled"** with a **+N blocks** count that keeps climbing — or posting or taking an offer was refused with a message that the wallet is locked — the cause is almost always the same: the node's coin wallet is **encrypted and locked**. A locked wallet can still read its balance, but it can't sign the funding transaction, so your side of the swap can't lock.

To fix it:

1. **Unlock the node wallet.** Run `walletpassphrase` on the coin's node, and keep it unlocked until the swap completes.
2. **Let the swap self-heal.** For a **Standard (HTLC)** swap the engine retries the fund automatically on every tick, so once the wallet is open the swap funds itself on the next pass — there's nothing more to press. The retry is safe to repeat; it won't double-fund.
3. **Or fund it manually.** If you'd rather not wait, use the **fund** action on the swap's card. A manual fund now also notifies the counterparty, exactly like the automatic path, so the other side picks up from where you left off.

   Note — A swap completes purely by **chain-watching** even if relay messages are missed: once your funding is on chain, the engine settles it from what it sees on the blockchain. The unlock is the only thing it needs from you.

18.9 I can't take an offer

If the **Take offer** button is disabled, or a take fails, the offer isn't available to you right now. Common reasons:

- **It's your own offer.** You can't take an offer you posted — you can only **Withdraw** it.
- **A leg is down.** If one of the two coins involved isn't connected, the take is blocked until you fix that coin (**Settings** → **Coins**).
- **It expired.** Offers have a "valid for" window. An expired offer can't be taken; refresh the board and look for a current one.

- **It was already taken or withdrawn.** Someone else may have taken it first, or the maker pulled it. The offer's state chip tells you which.
- **Your take arrived too late.** If the maker's Satchel was unreachable for a while (their computer was asleep, offline, or its engine restarting) and your take sat waiting for more than about 15 minutes, the maker's engine treats it as abandoned and quietly drops it rather than serving a handshake nobody is watching anymore — the offer stays live rather than being burned on a dead take. If your own "initiating" card seems to have vanished without turning into a swap, this is almost always why: just try the offer again (or a fresh one) once you can see the maker is back online.

18.10 Windows SmartScreen warning

The first time you run Satchel on Windows, **SmartScreen** may show a blue "Windows protected your PC" box. This appears for new applications that aren't yet widely recognised by Microsoft — it isn't a sign anything is wrong with the download.

To proceed, click **More info**, then **Run anyway**.

> **Tip** — Only do this for a copy of Satchel you downloaded from the official project repository. If in doubt about where your download came from, stop and verify the source first — see the chapter *"Where to Get Help"* for the official links.

18.11 Getting diagnostics for a developer

If a swap misbehaves and you want a developer to look into it, you don't have to go hunting through log files. Open the swap — either its row on the **Swaps** page (expand it) or its card in the active-swaps dock — and press **Dump logs**. This copies a diagnostics bundle for that one swap to your clipboard: the swap's record plus the engine log lines about it. Paste it straight into your bug report.

> **Tip** — The bundle is **secret-free**: it never contains your seed, recovery phrase, swap preimage, or nonces, so it's safe to share. See *"Tracking Your Swaps"* for more on the **Dump logs** button.

18.12 Still stuck?

If none of the above fixes it, the chapter *"Where to Get Help"* lists the project repository and community channels, and explains how to file a useful bug report — including which logs to attach (and how to be sure they contain **no** seed or passphrase).

Chapter 19

Frequently Asked Questions

Short answers to the questions people ask most. Each one points to a fuller chapter if you want the detail.

19.1 Is there a fee?

No platform fee. Satchel and the project behind it take *nothing* from your trades — there is no commission, no spread, no membership cost. The only cost you pay is the ordinary **mining fee** (also called a network or transaction fee) that every blockchain charges to confirm a transaction. That fee goes to the blockchain's miners, never to us, and you'd pay it for any on-chain transaction.

19.2 Who holds my coins?

You do — always. At no point does Satchel, a noticeboard, or your trading partner hold your funds. Your coins stay in wallets you control, on nodes you run, secured by keys only you have. During a swap they're briefly locked into an all-or-nothing arrangement enforced by the blockchain itself — not handed to anyone.

19.3 Do I need to run nodes?

No — it's your choice per coin. Each coin connects either to **your own node** (RPC — the node's wallet funds swaps, maximum sovereignty) or to **Electrum servers** (no node: chain data comes from the servers and the wallet lives on your Pact seed — the servers never see your keys). Mixing is normal: a BTCX miner already running a node keeps it, and connects Bitcoin via Electrum to skip the multi-day node sync. Both are set up in the chapter *"Setting Up Your Coins"*.

> **Note** — On mainnet an Electrum coin requires **at least two independent servers**, which cross-check each other's view of the chain.

19.4 Is it safe to close the app?

Not during a swap. While a swap is in progress, keep Satchel and your nodes running — the engine needs to be awake to broadcast each step (including your refund) on time. If you try to quit mid-swap, Satchel warns you and offers **Keep running** in the background; take that. The same warning guards **Quit** in the tray icon's menu, so no exit path skips it — and while Satchel runs in the background, swap milestones reach you as desktop notifications. When no swap is active, you can close it freely. See *"Backup, Seeds & Safety"*.

19.5 What if the other side disappears?

You get refunded, automatically. Every swap has a *timelock* — a built-in deadline. If your counterparty walks away and the swap can't complete, your locked funds become refundable to you once that deadline passes, with no cooperation needed from them. The chapter *"Understanding Atomic Swaps"* explains why this is guaranteed.

19.6 What's the difference between public and private offers?

A **public offer** is posted to a noticeboard (Nostr or a Corkboard) where anyone can browse and take it — that's the **Corkboard** screen. A **private offer** is a *slip* you create and hand directly to one person (paste it into a chat, say); it isn't posted publicly, and only someone holding the slip can take it. Use public offers to find traders; use private offers to trade with a specific friend. Both settle with the same atomic-swap safety.

19.7 Can I trade on mainnet?

Yes. Both swap types — **Standard (HTLC)** and **Private (Taproot)** — are reviewed and running on real mainnet today.

> **Warning** — You alone hold your keys. Safeguard your recovery phrase, and keep Satchel and your nodes running until a swap completes so the engine can finish or refund on time.

19.8 What coins are supported?

The first supported pair is **BTCX** ↔ **BTC** (BTCX is Bitcoin-PoCX). **Litecoin (LTC)** was the first added third coin. The list grows over time: coins are defined in a plain text file (`coins.toml`) that ships beside the app, so new ones can be added without rebuilding Satchel. **Settings** → **Coins** shows what's available and which pairs are ready to trade.

19.9 Where are my keys and seed stored?

On your own machine, never in the cloud. Your recovery phrase and private keys are held locally by the engine and, if you chose a passphrase, stored encrypted. Satchel never sends them to any server, noticeboard, or person — and your settings file (`satchel.json`) never contains them. The chapter *"Backup, Seeds & Safety"* covers this in full, including the two golden rules for your recovery phrase.

19.10 Will anyone from the project ask for my recovery phrase?

Never. No genuine support person and no one from this project will *ever* ask for your recovery phrase. Anyone who does is trying to steal from you. Satchel only asks for it on its own first-run screen — never on a website, chat, or form.

Chapter 20

Glossary

Plain-language definitions of the terms you'll meet in Satchel and in this handbook. Where a term has its own chapter, that's the place to go for more.

Adaptor swap — See *Taproot swap*. The technical name for the **Private (Taproot)** swap method, where the two payments are linked using a hidden adaptor secret.

Atomic swap — A trade between two cryptocurrencies that is all-or-nothing: either both sides receive their coins or neither does, with no middleman holding funds. The blockchain itself enforces it. See *"Understanding Atomic Swaps"*.

Base / quote — The two sides of a trading pair. In **BTCX/BTC**, BTCX is the *base* (the thing being priced) and BTC is the *quote* (what it's priced in). The Corkboard uses this convention to lay out bids and asks.

BTCX / PoCX — **BTCX** is the ticker for **Bitcoin-PoCX** (PoCX = Proof-of- Capacity eXtended), the first coin Satchel trades. BTCX ↔ BTC is the first supported pair.

Confirmation — A blockchain's way of recording that a transaction is settled. Each new block adds one confirmation; more confirmations mean the transaction is more firmly final. Swaps wait for a set number of confirmations before proceeding.

Corkboard — A self-hostable *noticeboard* a community can run on its own server. You point Satchel at its web address to post and browse offers there. See *"Trading over Nostr & Corkboard"*.

Engine (pactd) — The local program that does the real swap work on your machine: holding your keys for the session, building and watching swaps, and auto-refunding if needed. Satchel's UI drives it; you'll see it called "the engine." Its technical name is `pactd`.

HTLC — *Hash Time-Locked Contract*: the classic, well-proven way to build an atomic swap, labelled **Standard (HTLC)** on offers. Recognisable on the blockchain as a swap.

Maker / taker — The *maker* is the person who posts an offer; the *taker* is the person who accepts it. You can be either, depending on whether you post or take.

Merchant — A trading identity in Satchel: one seed, one set of keys, one data directory. You can have several merchants (separate identities) and switch between them in the Merchant Manager.

Mining fee — The small fee every blockchain charges to confirm a transaction, paid to miners — not to Satchel. The only cost of a swap, beyond the coins themselves.

Node — The software that connects you to a cryptocurrency's network and holds your wallet for that coin (for example, Bitcoin Core for BTC). Satchel talks to the nodes you run. See *"Setting Up Your Coins"*.

Nostr — An open, public messaging network of independent servers (*relays*). Satchel's default *noticeboard*, prewired with six relays. No setup needed. See *"Trading over Nostr & Corkboard"*.

Noticeboard — A public place where offers are posted and browsed. Satchel supports two kinds: *Nostr* and *Corkboard*. A noticeboard never holds coins or keys and can't match or execute trades.

Offer — A signed statement of a trade you're willing to make: "I'll give X for Y, valid for this long." Public offers go on a noticeboard; private offers become *slips*.

Pair — Two coins that can be traded against each other, for example BTCX/BTC. **Settings** → **Coins** shows which pairs are ready to trade.

Passphrase — An optional password that encrypts the copy of your seed on this computer. It protects against someone using your machine — but it is *not* your recovery phrase and *not* a backup. Forget it and you must restore from your recovery phrase. See *"Backup, Seeds & Safety"*.

Recovery phrase (seed / seed phrase) — The list of 12 or 24 words that *is* your wallet. Write it down offline; never share it. It restores all your funds on any machine. The single most important thing to protect. See *"Backup, Seeds & Safety"*.

Redeem — Claiming your side of a completed swap — taking the coins you traded for, once the linked payments unlock.

Refund — Getting your own locked funds back when a swap doesn't complete. The *timelock* makes refunds automatic and guaranteed after its deadline.

Rescue — Satchel's safety net for a machine that dies mid-swap: an encrypted, self-only backup of your in-flight swaps rides along on the relays, so a new machine holding just your recovery phrase can find and finish them. Always gated behind an explicit confirmation — Satchel only ever warns you it found one, never resumes it silently. See *"Backup, Seeds & Safety"*.

Relay — One server on the *Nostr* network. Satchel posts to several at once so no single relay can silence you. The header's relay dot shows whether any are reachable.

RPC — *Remote Procedure Call*: the technical channel Satchel's engine uses to talk to your nodes. You'll meet the term only when entering a node's connection details (RPC host, port, credentials) in **Settings** → **Coins**.

Slip — A private, unposted offer in text form (it starts with `pactoffer1:`). You hand it directly to one person — paste it into a chat — and only someone holding it can take it.

Taker — See *Maker / taker*.

Taproot swap — The newer swap method, labelled **Private (Taproot)**. When it completes smoothly it looks like an ordinary transaction on the blockchain, giving better privacy. Also called an *adaptor swap*. See *"Understanding Atomic Swaps"*.

Timelock — A built-in deadline on the funds locked in a swap. If the swap doesn't complete by then, your funds become refundable to you automatically. The safety net behind every swap. You pick a **Short / Medium / Long** preset when trading.

Trustless — Describes a trade where you don't have to trust the other person or any middleman, because the rules are enforced by the blockchain itself. The whole point of atomic swaps.

Chapter 21

Where to Get Help

Stuck on something this handbook didn't cover, or found a bug? Here's where to turn — and one safety reminder that's worth repeating before anything else.

> **Warning** — **No one from this project will ever ask for your recovery phrase**, your passphrase, or your private keys — not in a chat, not in an "support" message, not anywhere. Anyone who asks is trying to steal from you. Genuine help never needs your seed. See *"Backup, Seeds & Safety"*.

21.1 The project repository

The home of Satchel is its public code repository:

> **github.com/PoC-Consortium/satchel**

There you'll find the latest releases and download links, release notes, the issue tracker, and the source code itself. When you want the official build, this is the place to get it — and the only place to trust a download from.

> **Tip** — Satchel can also tell you when a new version exists: check **Settings** → **About**, which shows your version and an update notice when one's available.

21.2 Filing an issue

If something's broken or behaving oddly, a good bug report helps it get fixed fast. Open an issue on the repository's issue tracker, and include:

1. **Your Satchel version** — from **Settings** → **About** (for example, 0.1).
2. **Which network** you were on (mainnet, testnet, or regtest) and which **coins and pair** were involved.
3. **What you did, what you expected, and what actually happened** — step by step.
4. **Relevant logs**, if you have them — they often pin down the cause quickly.

Warning — Before you paste any log, configuration, or screenshot into a bug report, **double-check it contains no recovery phrase, passphrase, or private key.** Satchel keeps secrets out of its logs by design, but always glance over anything you share. When in doubt, leave it out.

Note — A first quick stop before filing: the chapter *"Troubleshooting"* covers the most common snags — a node that won't connect, an empty Corkboard, a swap that looks stuck — and often saves you the trip.

21.3 Community channels

Beyond the repository, the project's community is the place to ask questions, share what you're building, and get a hand from other users. The repository's README and release notes link to the current community channels — check there for the up-to-date list, since the venues can change as the project grows.

When you ask for help, the same etiquette applies as for bug reports: describe what you're seeing clearly, mention your version and network — and never share your seed.

21.4 For developers: the Pact handbook

This book is the *user's* guide. If you're a developer or integrator who wants the machinery underneath — how swaps are constructed transaction by transaction, the exact protocol messages, the engine's RPC interface, key derivation, and the cryptography — that all lives in the companion book:

Pact Developer & Integrator Handbook

It's dense and precise, written for people comfortable with Bitcoin and code. If you've ever wondered *exactly* how the all-or-nothing guarantee is enforced, or you want to build on top of the engine, that's your reference.

21.5 A final word

Satchel is live, and the project is small and moving fast. Your reports genuinely help shape what comes next. Thanks for trading carefully, keeping your recovery phrase safe, and helping the project improve.